
SOFTWARE ENGINEERING

Comprehensive E-Book Notes

IIT Madras BS Degree Programme

- Week 1-5 Detailed Notes — Every Topic Explained Thoroughly
 - Version Control (Git/GitHub) + Issue Tracking & Code Review
 - 40+ Practice MCQs from Both Previous Year Papers
 - Diagrams, Flowcharts & Visual Aids Throughout
 - Exam Tips, Common Traps & What to Watch Out For
 - Quick Revision Cheat Sheet & Topic-Wise Summary
 - Analogies & Real-World Examples to Make Concepts Click
 - COCOMO Worked Examples with Step-by-Step Solutions
-

Made by Anmol Kansal with AI help

Dr. Sridhar Iyer (IIT Bombay) & Dr. Prajish Prasad (FLAME University)

TABLE OF CONTENTS

Week 1	Deconstructing the Software Development Process Software Components & Modules • SDLC Phases (Requirements, Design, Development, Testing, Maintenance) • Client Types (External, Internal, Another Software) • Testing Types (Unit, Integration, Alpha, Beta) • SDLC Models (Waterfall, Prototype, Spiral) • Agile Manifesto & Practices • When to use Agile vs Plan-and-Document
Week 2	Software Requirements Requirement Gathering & Analysis • User Classification (Primary, Secondary, Tertiary) • 5 Gathering Techniques (Questionnaires, Interviews, Focus Groups, Observations, Documentation) • Interview Types • Requirement Problems (Incomplete, Ambiguous, Inconsistent, Implicit) • Functional vs Non-Functional Requirements • SRS Document Structure • BDD & User Stories • SMART Criteria
Week 3	Software User Interfaces Interaction Design Activities • Usability Goals (Effectiveness, Efficiency, Safety, Learnability, Memorability) • User Experience Goals • Prototyping (Storyboard, Paper, Digital Mock-up, Interactive) • Nielsen's 10 Heuristics in 3 Categories (Understanding, Action, Feedback) • Heuristic Evaluation Process
Week 4	Software Project Management PM Responsibilities • Estimation (Expert Judgement, Delphi, COCOMO) • COCOMO Model Deep Dive (Organic, Semi-detached, Embedded) • Cost Drivers & EAF • Scheduling (WBS, Activity Networks, Gantt Charts, Critical Path) • Risk Management (Technical, Project, Business) • Risk Assessment • Agile PM (Scrum, Sprints, Velocity, Story Points)
Week 5	Software Design Design Outcomes • Good Design Attributes • Coupling Types (Content, Common, Control, Stamp, Data) • Cohesion Types (Coincidental, Procedural, Communicational, Sequential, Functional) • OOP Concepts (Objects, Classes, Encapsulation) • Class Relationships • UML Diagrams • Architecture Patterns (P2P, Client-Server, MVC) • SOLID Principles • Design Patterns (Facade, Adapter, Builder, Strategy) • Cyclomatic Complexity
Bonus	Version Control, Issue Tracking & More VCS (Centralized vs Distributed) • Git (Workflow, Commands, States) • Branching, Merging, Rebase • GitHub • Issue Tracking • Code Review • Pull Requests • Pair Programming • IDEs vs Text Editors • Debugging Techniques (Black Box, Forward, Backward)
Exam	Practice MCQs, Tips & Cheat Sheet 40+ Practice MCQs with Explanations • Previous Year Paper Analysis • Common Traps • Quick Revision Cheat Sheet • Exam Strategy

WEEK 1

Deconstructing the Software Development Process

■ Topics Covered This Week

1. Thinking of Software in terms of Components (Modules)
2. Software Development Process: Requirements → Design → Development → Testing → Maintenance
3. Types of Clients: External User, Internal, Another Software
4. Testing: Unit, Integration, Acceptance, Alpha, Beta
5. SDLC Models: Waterfall, Prototype, Spiral
6. Agile Manifesto, Practices (XP, Scrum, Kanban), When to use which

1.1 Software Components & Modules

Large software systems like Amazon, Google, or banking platforms are not built as one monolithic piece. Instead, they are divided into **modules** — separately addressable components that are integrated to satisfy requirements.

Consider Amazon as an example. It has many components working together:

- **Inventory Management System:** Tracks stock levels, updates based on purchases and seasonal trends. Your homepage is customized based on your shopping and viewing history.
- **Payment Gateway:** Handles the flow of payment — customer submits order → store contacts gateway → gateway confirms payment → merchant payment processor settles the payment.
- **Recommendation Engine:** Suggests products based on browsing and purchase history.
- **Order Tracking:** Real-time updates on order status and delivery.

■ ANALOGY

Think of a car — the engine, transmission, brakes, electrical system, and body are all separate modules that are designed independently but work together as a whole. If the braking system has a problem, you fix it without rebuilding the engine. Software works the same way — each module handles a specific responsibility.

■ REMEMBER

Software can be divided into separately addressable components called **MODULES** that are integrated to satisfy requirements. This is the fundamental principle of modular software design.

1.2 The Software Development Process

Before you start coding, you need a structured process. The software development process consists of five key phases, each building on the previous one:



Phase 1: Requirement Specification

This is the **first step** in creating any new software component. Before writing any code, you must:

- **Study existing components** of the system to understand how the new component will interact with them
- **Look at similar systems** (competitive analysis) to understand features and user expectations

- **Define requirements** — goals the implemented system should have, catering to the needs of clients

Who are the clients? Think about who is going to use your software, for what purpose, and in what way. Clients can be of three types:

Client Type	Definition	Real-World Example	Exam Example
External User	A person OUTSIDE your organization who uses the software	Bank customers using mobile banking app; online shoppers on Amazon	Prospective students using university application portal
Internal to Company	A person INSIDE your organization who uses the system as part of their work	Faculty using internal dashboard; HR team using employee portal	Company's own developers using internal SDLC tool
Another Software	Not a human — another software system that communicates programmatically (via APIs)	Payment gateway interfacing with e-commerce system; library system syncing with digital archive	Razorpay API connecting to an online store

■ EXAM ALERT

Client type questions are very common! Key traps:

- 'Prospective students' applying to university = EXTERNAL (not yet part of org)
- 'Enrolled students' accessing content = could be secondary/internal
- 'Library system syncing with archive' = ANOTHER SOFTWARE (system-to-system)
- 'Company builds tool for its OWN developers' = INTERNAL users

Phase 2: Software Design

Once requirements are gathered, the design phase creates a **big picture view** of the software system. It provides structure and ensures that developers have a shared understanding before they start coding.

- Why not just start coding? Without a design, different developers may have different ideas about implementation, causing **integration issues**.
- Without a big picture view, adding new features becomes extremely difficult.
- The design phase produces: component breakdown, interfaces between components, data structures, and algorithms.

Phase 3: Development

The actual coding phase. Key characteristics:

- Code is written based on requirements AND the design
- Work is usually **distributed** among team members — different developers work on different modules
- Developer documentation and precise **interface definitions** are created so modules can work together
- Each developer implements their assigned module's internal logic while respecting the interfaces

Phase 4: Testing

Testing ensures the software behaves according to requirements. **Why is this critical?** In 2002, software bugs cost the US economy \$59.5 billion annually. By 2016, this figure reached \$1.1 trillion. Failure to address bugs can cause severe catastrophes (e.g., Therac-25 radiation machine).

Testing Type	What It Tests	Who Does It	When
Unit Testing	Individual components/functions in isolation	Developers	During development
Integration Testing	How components work TOGETHER	Dev/QA team	After unit testing
Acceptance Testing	Does system meet client requirements?	Client/stakeholders	Before final delivery
Alpha Testing	Full system in lab/staging environment	INTERNAL employees	Before public release

Beta Testing	Full system in real-world conditions	EXTERNAL actual users	Before final release
---------------------	--------------------------------------	-----------------------	----------------------

■ EXAM ALERT

Alpha vs Beta is asked in EVERY paper!

- Alpha = INTERNAL testers at developer's site, in a controlled environment
- Beta = EXTERNAL/actual users in REAL-WORLD environments
- Key phrases: 'actual users' + 'real-world scenarios' + 'before final release' = BETA
- 'Internal employees' + 'lab/staging' = ALPHA
- A complete product tested by real users for real-world feedback = BETA

Phase 5: Maintenance

After the software is released/deployed, the maintenance phase begins. This is often the **longest phase** in the software lifecycle.

- **Monitor** what users are doing and how they use the software
- **Change code** for upgrades, updates, and patches
- **Add new features** based on user feedback and market demands
- **Fix bugs** reported by users after deployment

■ EXAM ALERT

If a bug is found AFTER deployment by actual users/customers = MAINTENANCE phase.

If a bug is found BEFORE release during internal testing = TESTING phase.

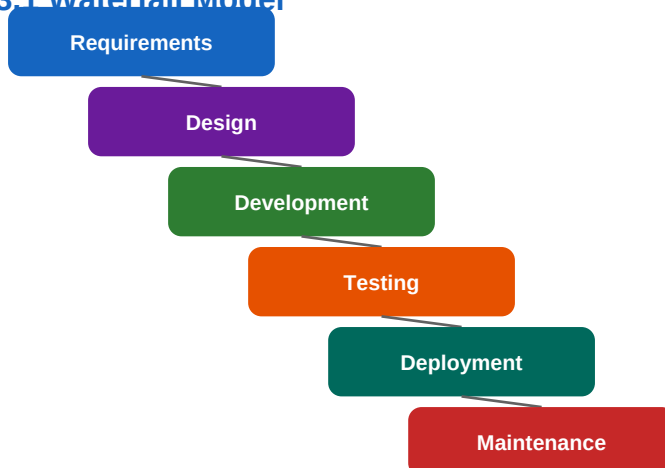
This distinction appears in almost every paper. Example: 'Food delivery app shared with customers. They report incorrect delivery tracking.' = Maintenance phase.

1.3 Software Development Lifecycle Models

Different organizations follow different approaches to move through the SDLC phases. These approaches are called **SDLC models** or **software process models**. They fall into two broad perspectives:

- **Plan and Document Perspective:** Heavy upfront planning, detailed documentation (Waterfall, Spiral)
- **Agile Perspective:** Lightweight, iterative, continuous feedback (Scrum, XP, Kanban)

1.3.1 Waterfall Model



WATERFALL MODEL

- Each phase completes fully before next
- No going back to previous phase
- Client sees product only at the END
- Takes 6-18 months for 1 cycle
- High cost if changes needed later
- Works only if requirements are FIXED

The Waterfall Model follows a **strict sequential** process. Each phase must be completed fully before the next phase begins. There is **no going back** to a previous phase.

Drawbacks of the Waterfall Model

- **Increase in cost and time** if changes are required later — going back is extremely expensive

- **Clients may not know what they need** at the beginning — they can only articulate requirements after seeing something
- **Designers may not know** which design is most feasible/usable until they test with users
- **Very long cycles** — typically 6-18 months for one complete cycle
- **Late feedback** — client sees the final product only at the end, leading to potential dissatisfaction

■ EXAM ALERT

When you see: 'strict sequential process' + 'long cycle (12 months)' + 'client dissatisfied at final release' + 'misunderstood requirements' → This is classic WATERFALL and its problems!

1.3.2 Prototype Model

To address the Waterfall's drawback of late feedback, the Prototype Model builds a **working prototype** before developing the actual software.

- **Prototype is usually NOT used later** — it's thrown away after feedback is gathered
- **Advantages:** Useful when exact form of solution and technical issues are unclear; great for getting customer feedback early
- **Disadvantages:** Increased development costs (building something that's thrown away); bugs can still appear later in the actual development

1.3.3 Spiral Model

The Spiral Model **incrementally builds** the software through cycles, getting feedback and refining at each cycle. It combines elements of both Waterfall and Prototyping.

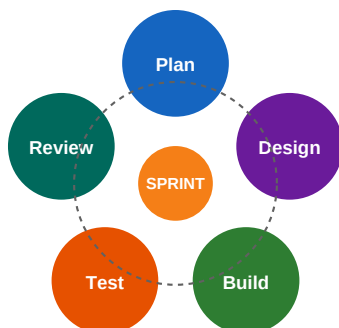
- Each cycle involves: Planning → Risk Analysis → Development → Evaluation
- Heavy emphasis on **risk analysis** at each spiral
- Software grows incrementally with each cycle

■ REMEMBER

Waterfall = sequential, no going back. Prototype = build throwaway prototype first. Spiral = iterative cycles with risk analysis. Each model tries to address the previous one's limitations.

1.4 The Agile Perspective

Studies of software projects paint a troubling picture. In Johnson's 1995 study, a large percentage of projects were terminated, abandoned, or delivered late/over budget. By 2013, only about 39% of projects were on time and on budget. The Agile approach emerged to address these systematic failures.



AGILE / ITERATIVE

- Short sprints (1-2 weeks each)
- Continuous feedback from client
- Requirements refined each sprint
- ✓ Adapts to changes quickly
- ✓ Working software after every sprint
- ✓ Team: 4-9 people preferred

The 4 Agile Manifesto Values

The Agile Manifesto (2001) established four core values. **The left side is preferred over the right side**, but the right side is not ignored:

WE VALUE THIS MORE ✓	OVER THIS (still has value, but less)
Individuals and interactions	Processes and tools
Working software	Comprehensive documentation
Customer collaboration	Contract negotiation
Responding to change	Following a plan

■ EXAM ALERT

THE BIGGEST TRAP IN EXAMS: Options often REVERSE the Agile values!

- WRONG: 'Processes and tools over individuals and interactions'
- WRONG: 'Following a plan over responding to change'
- These are OPPOSITE of what Agile says! Always check which side is 'over' which.

Example Q: Client wants email alerts mid-development. Which principle?

→ 'Customer collaboration over contract negotiation' (not 'contract over collaboration')

Agile Approaches & Practices

Approaches: Extreme Programming (XP), Scrum, Kanban

Key Practices:

- **User Stories:** Short descriptions of what a user wants (covered in detail in Week 2)
- **Sprints:** Short, time-boxed iterations (typically 1-2 weeks)
- **Scrum Stand-up Meetings:** Daily 15-minute meetings where each member answers: What did I do yesterday? What will I do today? What blocks me?
- **Test-Driven Development (TDD):** Write tests BEFORE writing the actual code

Agile Philosophy: Rather than just following specific approaches and practices mechanically, Agile is about adhering to the **broad philosophy** of collaboration, adaptability, and delivering working software.

When to Use Agile vs Plan-and-Document?

Sommerville's checklist: A 'No' answer suggests Agile; a 'Yes' suggests Plan-and-Document:

Question	No → Agile	Yes → Plan & Doc
Is specification required?	Requirements evolving	Requirements fixed upfront
Are customers unavailable?	Customer always available	Customer not accessible
Is the system large/complex?	Small/medium system	Large, complex system
Will it have a long product lifetime?	Short/medium lifecycle	Long lifecycle (10+ years)
Is team geographically distributed?	Co-located team	Distributed team
Documentation-oriented culture?	Lean culture	Heavy documentation culture
Subject to regulation?	No regulatory constraints	Safety-critical, regulated

■ TIP

Quick rule of thumb:

- Small team + evolving requirements + customer available = AGILE
- Large team + fixed requirements + safety-critical + regulated = PLAN AND DOCUMENT

WEEK 2

Software Requirements

■ Topics Covered This Week

1. Importance of Requirement Gathering and Analysis
2. User Classification: Primary, Secondary, Tertiary Users
3. Requirement Gathering Techniques (5 methods with examples)
4. Interview Types: Structured, Semi-structured, Unstructured
5. Types of Requirement Problems: Incomplete, Ambiguous, Inconsistent, Implicit
6. Functional vs Non-Functional Requirements (with NFR categories)
7. SRS Document: Structure, Sections, Advantages, Drawbacks
8. Behaviour Driven Design (BDD) and User Stories
9. SMART Criteria for User Stories: Specific, Measurable, Achievable, Relevant, Time-boxed

2.1 Requirement Gathering and Analysis

Requirements identification and analysis is **crucial** because:

- **Developers understand what customers want:** Without this, you're building blindly
- **Customers come to an agreement:** Different stakeholders may have different expectations
- **Reduces cost:** If requirements are not understood properly initially, the cost and number of iterations increases dramatically

■ ANALOGY

Imagine building a house without blueprints. The electrician puts wires where they think best, the plumber does the same — and when you try to assemble everything, nothing connects. Requirement gathering is like creating detailed blueprints that everyone agrees on before construction starts.

2.1.1 User Classification

Understanding WHO will use the software is the first step in gathering requirements. Users are classified into three categories. The example used in the course is the **Amazon Seller Portal**:

User Type	Definition	Key Characteristic	Amazon Seller Portal Example	Exam Example (Education Platform)
Primary	Frequent, direct users of the system	System is CENTRAL to their daily workflow	Independent sellers, Sales teams, Authors/Publishers	Professors (upload materials, manage courses)
Secondary	Use system regularly but as consumers, not creators	Use through an intermediary or consume content	Buyers browsing products listed by sellers	Students (access/consume content)
Tertiary	Do not use the software directly; affected by its introduction or influence its purchase	Interact only OCCASIONALLY or indirectly	Shipping company managers, Banks	Accreditation body ('occasionally' reviews reports)

■ EXAM ALERT

User classification is in EVERY exam paper. Key traps:

- 'Occasionally' strongly signals TERTIARY
- Don't think students are primary just because there are MANY of them — frequency and depth of use matters
- Primary users: system is CENTRAL to their work (professors managing courses)
- Secondary users: CONSUME what primary users create (students accessing content)
- Tertiary: only affected indirectly or involved occasionally

This concept has some flaws, it doesn't match with our course content so kindly review it yourself.

2.2 Requirement Gathering Techniques

Once you've identified your users, you need systematic methods to gather their requirements. There are **five main techniques**, each suited for different situations:

2.2.1 Questionnaires

- Series of questions designed to elicit **specific information** from users
- Good for getting answers to **specific questions from a LARGE group** of people
- Example: Asking sales team managers — 'What percentage of your inventory do you sell online?' 'What difficulties do you encounter selling online?'
- Usually used **in conjunction** with other techniques (not alone)

2.2.2 Interviews

Asking a set of questions — face-to-face, telephonic, or online. Three types:

Type	Predefined Questions?	Can Deviate?	Best For	Key Identifier
Structured	YES — all questions fixed	NO — same order, same Qs for everyone	Consistent data collection	'Predefined set' + 'fixed order'
Semi-structured	YES — has a guide/list	YES — can ask follow-ups	Balance of structure and exploration	'Has a guide BUT allows flexibility'
Unstructured	NO — no predefined questions	Fully adaptive to conversation	Deep exploration, open-ended	'No predefined set' + 'open-ended' + 'adapts'

■ EXAM ALERT

Interview type is asked frequently. The TRAP:

- Semi-structured HAS a predefined list but allows some flexibility
- Unstructured has NO predefined list at all
- Key phrase: 'INSTEAD OF following a predefined set' + 'open-ended' + 'adapts follow-ups' = UNSTRUCTURED
- If it says 'has a guide but can deviate' = SEMI-STRUCTURED

2.2.3 Focus Groups

- Get a **group of stakeholders** to discuss issues and requirements together
- Overcomes drawback of interviews (which give only ONE perspective)
- **Advantages:** Gaining consensus; highlighting areas of conflict and disagreement
- Example: Sales managers from different industries discuss different expectations — mobile sales wants launch-day summary, furniture wants 30-day summary → Requirement: Sales summary for specific day / last x days

2.2.4 Naturalistic Observations

- Spending time with stakeholders in their **natural setting**, observing day-to-day tasks
- Shadowing a stakeholder, making notes, asking questions while they work

- Example: Observing how people sell in physical shops → insights like shopkeeper recommendations, customer feedback patterns

2.2.5 Documentation (Document Analysis)

- Studying **existing written materials**: procedures, rules, steps, regulations, policies, competitor FAQs
- Example: Reading bank regulations to understand how to add sellers' bank accounts, payment frequencies
- Example: Studying competitor FAQs and official policies for financial feature design

Technique	Best For	When to Use
Questionnaires	Specific answers from LARGE groups	Need quantifiable data from many people
Interviews	Exploring issues IN DEPTH	Need to understand individual perspectives
Focus Groups	Multiple viewpoints, CONSENSUS	Need to find agreement/disagreement among stakeholders
Naturalistic Observation	Understanding real CONTEXT	Need to see how people actually work (not what they say)
Documentation	PROCEDURES, regulations, standards	Need to understand existing rules, policies, written materials

■ EXAM ALERT

Documentation is the most commonly tested technique:

- 'Reading banking policies' = Documentation
- 'Studying competitor FAQs' = Documentation
- 'Reviewing official regulations' = Documentation
- 'Recorded procedures, rules, steps' = Documentation
- Don't confuse with Observation (watching people) or Interviews (asking people)

■ REMEMBER

Basic guidelines for requirement gathering:

- Focus on identifying stakeholder NEEDS
- Involve ALL stakeholder groups
- Use COMBINATION of techniques
- Run a pilot session if possible
- Be pragmatic — data gathering is expensive and time-consuming

2.3 Types of Requirement Problems

After gathering requirements, analysis is essential to identify problems. Four types of issues can arise:

Problem Type	Definition	Example	How to Identify
Incomplete	Requirement is missing information	'System should handle user data' — HOW? What data?	Requirement leaves questions unanswered
Ambiguous	Can be interpreted in MULTIPLE ways	'Good response time' — is that 1 second? 10 seconds?	Different people read it differently
Inconsistent	Two requirements DIRECTLY CONTRADICT each other	'Change password every 30 days' vs 'No changes more than once a year'	Both clear individually but CANNOT be satisfied simultaneously
Implicit	Assumed but NEVER explicitly stated	'Obviously users need internet' — but nobody wrote it down	Unstated assumptions

■ EXAM ALERT

Inconsistent is the most tested type:

- Both requirements are CLEAR individually — they just conflict
- 'Change every 30 days' = 12 changes/year vs 'no more than once a year' → INCONSISTENT
- If only ONE requirement is vague → Ambiguous
- If requirement is clear but missing details → Incomplete

2.4 Functional vs Non-Functional Requirements

This is one of the most important distinctions in requirements engineering:

	Functional Requirements	Non-Functional Requirements
What it describes	WHAT the system should DO	HOW WELL the system should BEHAVE
Focus	Features, behaviors, specific functions	Quality attributes, performance, constraints
Keywords	View, add, delete, notify, search, track	Within X seconds, support N users, 99.9% uptime
Example 1	Seller can add/edit/delete catalogue	Product appears in catalogue within 5 seconds
Example 2	View available doctors and slots	Support 500 simultaneous logins
Example 3	Notify user when appointment booked	Process payments within 3 seconds
Example 4	Track customer feedback	System available 99.9% of the time

Non-Functional Requirement Categories:

- **Reliability:** The extent to which a program behaves the same way over time in the same operating environment
- **Robustness:** The extent to which a program can recover from errors or unexpected input
- **Performance:** Response time, throughput, resource utilization
- **Portability:** Works across different platforms and operating systems
- **Security:** Protection of data, authentication, authorization

■ EXAM ALERT

How to classify:

- Does it describe WHAT the system does (a feature/action)? → FUNCTIONAL
- Does it describe HOW WELL it does it (quality/performance)? → NON-FUNCTIONAL
- Numbers like '500 users', '3 seconds', '99.9% uptime' usually = Non-functional
- Verbs like 'view', 'add', 'notify', 'search', 'track' usually = Functional

Common exam Q: R1='Support 500 logins'(NFR) R2='View doctors'(FR) R3='Notify user'(FR) R4='Process within 3 sec'(NFR)

2.5 Software Requirements Specification (SRS) Document

In the **Plan and Document perspective**, requirements gathered by the system analyst are organized into a formal **SRS Document**. This follows the IEEE 830 standard.

SRS Document Structure

Section 1 — Introduction: Purpose, Scope, Definitions/acronyms, References, Overview. Provides a broad outline and description of the software system.

Section 2 — Overall Description: Product perspective (how it fits in the larger ecosystem), Product functions (summary), User characteristics, Constraints, Assumptions and dependencies.

Section 3 — Specific Requirements: This is the most detailed section:

- **External Interface Requirements:** User interfaces (UI), Hardware interfaces, Software interfaces (connections between components), Communication interfaces

- **System Features:** Each feature has: Introduction/purpose, Stimulus/response sequence, Associated functional requirements
- **Performance Requirements, Design Constraints, Software System Attributes**

SRS Advantages

- Forms an **agreement** between customers and developers
- **Reduces future reworks** — everyone is on the same page
- Provides a **basis for estimating costs** and schedules
- **Facilitates future extensions** — new developers can read the SRS to understand the system

Drawback: Requires a LOT of documentation! Works well only if requirements are **fixed**. If requirements keep changing, the SRS becomes outdated quickly. This is where the Agile perspective (BDD/User Stories) comes in.

■ EXAM ALERT

Any option claiming 'eliminates need for documentation' is almost ALWAYS false. Documentation is always needed — Agile just makes it lighter.

2.6 Behaviour Driven Design (BDD) & User Stories

BDD asks questions about the **behaviour** of an application before and during development. Instead of writing heavy SRS documents, requirements are continuously refined to meet user expectations.

The BDD version of requirements is called **User Stories** — the smallest unit of work which can be done in 1 sprint.

User Story Template (Role-Feature-Benefit)

As a [type of user],

I want [an action],

So that [a benefit/value]

Example 1: As an independent seller, I want to view my inventory, So that I can take stock of products which are low in number.

Example 2: As an independent seller, I want to view my customers' feedback for each product, So that I can get a sense of pertinent issues in my products.

Benefits of User Stories

- **Lightweight:** No heavy documentation needed
- **Help plan and prioritize:** Developers can tell customers 'we'll deliver View Inventory in 2 weeks'
- **Focus on behaviour, not implementation:** Concentrate on WHAT the user needs, not HOW to code it
- **Enable conversation:** User can add details during discussion — 'I'd also like email notifications for 1-2 star ratings'

Drawbacks of User Stories:

- Continuous contact with users not always possible
- Don't scale well to very large projects or safety-critical applications

2.7 SMART Criteria for User Stories

Good user stories should be **SMART**. This is one of the most tested topics:

Letter	Criterion	What It Means	Example of VIOLATION	What the Fix Is
--------	-----------	---------------	----------------------	-----------------

S	Specific	Know EXACTLY what to implement	VAGUE: 'search for a product in catalogue' → SPECIFIC: 'search by TITLE in catalogue'	Add precise details
M	Measurable	Known expected results for given inputs	'Good response time' → 'product appears within 3 seconds'	Add quantifiable metrics
A	Achievable	Can be done in 1 sprint (1-2 weeks)	Story too big for one sprint → Subdivide into smaller stories	Break into sub-stories
R	Relevant	Has business value to stakeholders	Feature nobody asked for → Ask 'Why?' and 'So what?'	Validate with stakeholders
T	Time-boxed	Has a deadline/timeframe	No deadline specified → Stop if time budget exceeded	Set clear timeframe

■ EXAM ALERT

SMART questions appear in EVERY paper:

- 'Lacks a deadline or timeframe' → TIME-BOXED missing
- 'Cannot complete in one sprint' → not ACHIEVABLE → break down → makes it Achievable
- 'Increase cross-selling without saying by how much' → MEASURABLE missing
- 'User can search for product' (too vague, not specific enough) → SPECIFIC missing
- Time-boxed options: give up, divide story, reschedule what's left

■ TRICK

Quick SMART decision tree:

- Missing numbers/metrics? → Measurable
- Missing deadline? → Time-boxed
- Too vague/broad? → Specific
- Too big for a sprint? → Achievable
- No business value? → Relevant

WEEK 3

Software User Interfaces

■ Topics Covered This Week

1. Introduction to Interaction Design (4 activities)
2. Usability Goals: Effectiveness, Efficiency, Safety, Learnability, Memorability
3. User Experience Goals (subjective, emotional)
4. Prototyping: Storyboards → Paper → Digital Mock-ups → Interactive Prototypes
5. Nielsen's 10 Heuristics for UI Evaluation (grouped into 3 categories)
6. Heuristic Evaluation Process

3.1 Introduction to Interaction Design

Designing good user interfaces is critical — bad UI leads to user frustration, abandonment, and lost revenue. Interaction design involves four key activities:

- **1. Identifying needs and requirements** — understanding what users need from the interface
- **2. Developing alternative designs** that meet those requirements — don't settle on the first idea
- **3. Building interactive versions** (prototypes) — test before committing to full development
- **4. Evaluating** — assess the design with users and experts

3.2 Usability Goals

ISO 9241-11 defines usability as: *'The extent to which a product can be used by specified users to achieve specified goals with **effectiveness, efficiency and satisfaction** in a specified context of use.'*

Goal	Definition	Question It Answers	Real-World Example
Effectiveness	How GOOD a system is at doing what it's supposed to do	Can people accomplish their goals?	Google: type a query, get relevant results immediately
Efficiency	Common tasks through MINIMAL number of steps	How few steps to complete a task?	Amazon 1-click buy; minimal checkout flow
Safety	Protecting user from dangerous conditions or unwanted actions	Can errors be avoided/recovered?	Undo (Ctrl+Z), Trash recovery, confirmation dialogs, 'Save' and 'Close' placed far apart
Learnability	How EASY a system is to LEARN to use	Can new users get started quickly?	Step-by-step wizards, templates, tooltips, guided tours (Intro.js)
Memorability	How easy to REMEMBER how to use, once learned	Can returning users navigate without relearning?	Consistent layouts, familiar patterns across the app

■ EXAM ALERT

Learnability questions are common:

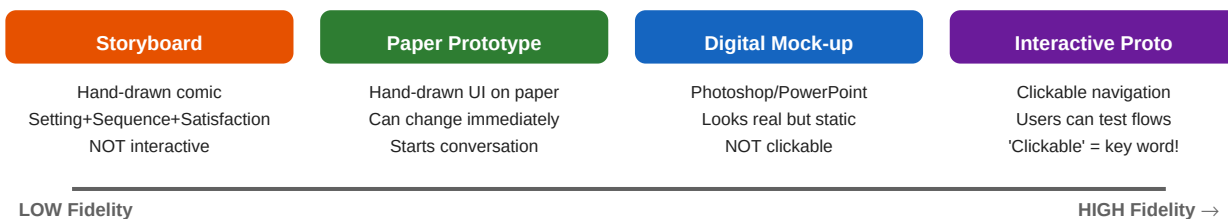
- Features that help FIRST-TIME users (wizards, templates, tutorials, tooltips) = LEARNABILITY
- Keyboard shortcuts help EXPERIENCED users = EFFICIENCY (not learnability!)
- Security features (strong passwords) = NOT learnability
- Tax filing software for first-time users: step-by-step wizard ✓, templates ✓ = Learnability
Undo shortcut = error recovery/efficiency, NOT learnability

User Experience Goals

Beyond usability (objective), UX goals are about **positive emotions** while using the software — satisfying, enjoyable, fun, helpful, entertaining, motivating, emotionally fulfilling, aesthetically pleasing. These are more **subjective** and measured from the user's perspective.

3.3 Prototyping

Before coding the UI, you should prototype to test ideas quickly. Why? Initial design ideas might not be the best; developing a full interface takes time, money, and effort; prototypes allow quick testing, feedback, iteration, and pivoting.



Prototype Type	Medium	Interactivity	Cost	Best For	Key Exam Identifier
Storyboard	Hand-drawn comic	NOT interactive	Very low	Understanding user TASKS and journey	Setting + Sequence + Satisfaction
Paper Prototype	Hand-drawn interface	Simulated by human	Low	Starting conversations about interactions	Hand-drawn, multiple paper pieces
Digital Mock-up	Photoshop, PowerPoint	Static images	Medium	Visual design, picky details	Looks real but NOT clickable
Interactive Prototype	Clickable web/app	Users can click & navigate	Higher	Testing real user flows	'CLICKABLE' is the key word!

■ EXAM ALERT

Prototype questions appear in every paper:

- 'Clickable PowerPoint images where users try placing an order' = INTERACTIVE PROTOTYPE
- The word 'clickable' is the DECIDING factor between digital mock-up and interactive
- If just images with no interaction → Digital mock-up
- If hand-drawn → Paper prototype
- If comic showing a scenario → Storyboard

Storyboard Details: A hand-drawn comic featuring three elements:

- **Setting:** Who is involved? What's the environment? What task is being accomplished?
- **Sequence:** What steps are involved? What leads someone to use the app?
- **Satisfaction:** What motivates people? What does the system enable them to accomplish?

3.4 Nielsen's Heuristic Evaluation

Heuristics are **rules of thumb** / guidelines derived from previous experience. They are used by experts to systematically evaluate a UI prototype. Nielsen's 10 heuristics are grouped into 3 categories:

UNDERSTANDING	ACTION	FEEDBACK
1. Consistency 2. Familiar Metaphors & Language 3. Clean & Functional Design	4. Freedom (Undo) 5. Flexibility 6. Recognition over Recall	7. Show Status 8. Prevent Errors 9. Error Recovery 10. Provide Help

Category 1: UNDERSTANDING

Heuristic 1: Consistency

Maintain **consistent layout, naming, and button placement** across the entire application. Same actions should look the same everywhere.

- Consistent button placement (e.g., 'Yes/No' dialogs always in same position)
- Consistent naming (don't call it 'Delete' in one place and 'Remove' in another)

Heuristic 2: Use Familiar Metaphors & Language

Use concepts users already know from the real world:

- Shopping 'Cart' icon for online shopping — everyone knows what a cart is
- 'Wallet' for mobile payments — familiar financial concept
- 'Folder' and 'File' icons on desktop — mimic physical office

Heuristic 3: Clean & Functional Design

Don't clutter the interface. Every element should serve a purpose. Example: Cleartrip's clean search interface vs cluttered, ad-heavy travel websites.

Category 2: ACTION

Heuristic 4: Freedom

Users should be able to undo actions and explore without fear:

- **Freedom to Undo:** Deleted a file? Recover from trash. Ctrl+Z to undo. WhatsApp 'delete for everyone'.
- **Freedom to Explore:** Users should feel safe clicking around without permanent consequences. Example: IRCTC's old design forced commitment early vs modern designs that let you explore freely.

Heuristic 5: Flexibility

- **Both experts AND new users** should be able to work efficiently
- Experts use **keyboard shortcuts** (Ctrl+C, Ctrl+V); beginners use menus
- **Personalization:** System tailors content for individual users (social media feeds)
- **Customization:** Users choose preferences (news app lets you select topics)

Heuristic 6: Recognition over Recall

Users find it easier to **recognize** something they've seen than to **recall** it from memory.

- Show buttons, menus, and navigation — don't force users to remember commands
- Example: Pre-GUI era required typing 'mv temp.txt new_folder/' — now you right-click and select Move
- Example: Google showing autocomplete suggestions as you type

Category 3: FEEDBACK

Heuristic 7: Show Status (Visibility of System Status)

Keep users informed about what's happening through appropriate feedback:

- Loading bars and progress indicators in browsers
- 'Storage 91% full' warning before you run out
- File download progress showing time remaining
- Order tracking showing: Placed → Preparing → Shipped → Out for Delivery → Delivered
- Provide next steps (e.g., 'Check your email to verify your account')

■ EXAM ALERT

'App fails to provide real-time updates about order/delivery progress' = SHOW STATUS violated.
This is one of the most tested heuristics. Any time a system fails to keep users informed about what's happening, it's Show Status.

Heuristic 8: Prevent Errors

Design the interface to prevent errors from happening in the first place:

- **Helpful constraints:** Date pickers prevent invalid dates; dropdown menus limit choices
- **Offer suggestions:** Google autocomplete reduces typos; search suggestions
- **Form validation:** Check input format before submission (email must have @)

Heuristic 9: Support Error Recovery

When errors DO happen, help users recover gracefully:

- **Make the problem clear:** Highlight which form field has an error, in red
- **Provide a solution:** 'Email format invalid — please enter a valid email address'
- **Provide an alternative:** Google's 'Did you mean: jakob nielsen?' when you type 'jacob neilson'

Heuristic 10: Provide Help

- **Ensure help is easy to search:** Search the menus (Alt+/,), keyboard shortcuts (Ctrl+/,)
- **Provide help in context:** Tooltip icons (?) next to form fields explaining what each field means
- Help documentation, tutorials, training materials

The Heuristic Evaluation Process

- Experts (not regular users) evaluate the prototype
- Do **multiple passes** through the interface
- Create a **list of issues** that violate specific design heuristics
- Prioritize issues by severity

■ TRICK

Heuristic matching cheat sheet for exams:

- No real-time progress/tracking updates → Show Status
- 'Did you mean...?' suggestion → Error Recovery
- Date picker preventing wrong dates → Prevent Errors
- Step-by-step wizard for new users → Learnability (usability goal, not heuristic)
- Keyboard shortcuts for experts → Flexibility
- Ctrl+Z, trash recovery → Freedom
- Search autocomplete → Prevent Errors (or Recognition over Recall)

WEEK 4

Software Project Management

■ Topics Covered This Week

1. Project Manager Responsibilities
2. Estimation: Empirical (Expert Judgement, Delphi) & Heuristic (COCOMO Model)
3. COCOMO Model Deep Dive: Project Types, Formulas, Cost Drivers, EAF
4. Worked COCOMO Examples (from both exam papers)
5. Scheduling: WBS, Activity Networks, Gantt Charts, Critical Path
6. Risk Management: Types (Technical, Project, Business), Assessment, Mitigation
7. Agile PM: Scrum Framework, Sprint Events, Velocity & Story Points

4.1 Project Manager Responsibilities

- **Main point of contact** between clients and the development team
- **Form and manage** the development team
- **Project Scheduling**: Create timelines, milestones, track progress
- **Project Estimation**: Estimate cost, effort, and schedule
- **Risk Management**: Identify, assess, and mitigate risks
- **Configuration Management**: Manage different versions and releases

4.2 Project Estimation

Estimation is critical for establishing **cost** and **schedule**, especially when bidding for projects.

Key Parameters

- **Size**: Number of lines of code. KLOC = thousands of lines of code (e.g., 4000 lines = 4 KLOC)
- **Effort**: Person-months — the effort one person can typically put in one month

■ ANALOGY

If a project requires 12 person-months: 4 developers can finish in 3 months, 2 developers in 6 months, or 1 developer in 12 months. But note: adding more people doesn't always speed things up proportionally (Brooks's Law).

4.2.1 Empirical Estimation Techniques

Expert Judgement

- Educated guess by experienced people
- Estimate cost of different components, then combine
- **Drawbacks**: Human errors, individual bias, optimistic estimates, may overlook factors

Delphi Technique

Addresses individual bias by using **multiple experts**:

- **Step 1**: Coordinator distributes SRS + estimation form to multiple experts
- **Step 2**: Experts submit their estimates independently (no discussion)

- **Step 3:** Coordinator prepares a summary and distributes it to all experts
- **Step 4:** Process iterated over several rounds until consensus
- **Step 5:** Coordinator compiles results and prepares final estimate

■ REMEMBER

92% of project managers use experience over formulas (Taylor, 2000). But COCOMO is heavily tested in exams, so learn it well!

4.3 COCOMO Estimation Model — Deep Dive

Constructive Cost Estimation Model proposed by Boehm in 1981. This is one of the MOST IMPORTANT topics for exams.

$$\text{Effort} = a \times (\text{SIZE in KLOC})^b$$

Type	a	b	Description
Organic	2.4	1.05	Small team, experienced, well-understood
Semi-detached	3.0	1.12	Mix experienced + inexperienced
Embedded	3.6	1.20	Coupled with hardware, complex, real-time

How to Determine Project Type

The description in the question gives clues. Look for these keywords:

Type	a	b	Team	Complexity	Keywords to Look For
Organic	2.4	1.05	Small, experienced	Low, well-understood domain	'Small team', 'experienced', 'well-understood', 'simple'
Semi-detached	3.0	1.12	Mix of exp + inexperienced	Moderate complexity	'Mix of experienced and new', 'moderate', 'business app'
Embedded	3.6	1.20	Large, highly skilled	Very high, HW integration	'Hardware integration', 'real-time', 'strict security', 'government'

Worked Example 1: Amazon Seller Portal (from slides)

Step 1: Determine type. Mix of experienced + inexperienced → **Semi-detached** (a=3.0, b=1.12)

Step 2: Estimate lines of code for each feature:

- Catalogue management: 300 LOC | Inventory: 300 LOC | Orders: 1000 LOC
- Payments: 800 LOC | Inventory tracking: 800 LOC | Sales: 500 LOC | Feedback: 300 LOC
- **Total = 4000 LOC = 4 KLOC**

Step 3: Calculate initial effort.

$$\text{Effort} = 3.0 \times (4)^{1.12} = 3.0 \times 5.28 = \mathbf{15.83 \text{ Person-Months}}$$

Step 4: Apply cost drivers. Selected drivers: RELY=1.40, DATA=1.08, AEXP=1.13, LEXP=0.95

$$\text{EAF} = 1.40 \times 1.08 \times 1.13 \times 0.95 = \mathbf{1.62}$$

Step 5: Final estimate. $15.83 \times 1.62 = \mathbf{25.65 \text{ Person-Months}}$

$$\text{Cost} = 16 \text{ months} \times \mathbf{75,000/\text{month}} = \mathbf{12 \text{ lakhs}}$$

Worked Example 2: Government Traffic Control (from Paper 2)

Step 1: 'Integration with hardware' + 'real-time processing' + 'strict security' + 'government' → **Embedded** (a=3.6, b=1.20)

Step 2: Source code = 1,00,000 lines. **CAREFUL:** Indian notation 1,00,000 = 100,000 = **100 KLOC** (NOT 1000!)

Step 3: Effort = $3.6 \times (100)^{1.20} = 3.6 \times 251.19 = 905 \text{ Person-Months}$

Step 4: Given EAF = 1.88

Step 5: Final = $905 \times 1.88 = 1702 \text{ Person-Months}$

■ EXAM ALERT

COCOMO is in EVERY exam paper (comprehension questions worth 6-9 marks):

1. FIRST determine project type from description → pick a and b
2. Calculate initial effort using formula
3. EAF = PRODUCT of all cost driver multipliers (multiply, DON'T add!)
4. Final effort = Initial × EAF
5. Indian notation trap: 1,00,000 = 100,000 = 100 KLOC
6. Cost driver > 1 means MORE effort needed; < 1 means LESS
7. 'Mix of experienced and new developers' = Semi-detached
8. 'Hardware integration + real-time + government' = Embedded

4.4 Project Scheduling

Creating a schedule involves these main activities:

- 1. Identify all major activities
- 2. Break down each activity into tasks (**Work Breakdown Structure - WBS**)
- 3. Determine dependencies among tasks
- 4. Estimate time durations for each task
- 5. Represent information (Activity Network, Gantt Chart)
- 6. Determine the **Critical Path** — longest chain of dependent tasks = minimum project duration
- 7. Allocate resources to tasks

Work Breakdown Structure (WBS)

A **tree-like structure** where: Root = project name, each node is broken into smaller activities (children), each leaf = a task assignable to a developer (~2 weeks per task).

Example: Seller Portal → (Requirements, Design, Development) → Design has (Component diagram, DB schema, UI) → Development has (Catalogue, Inventory, Orders, Payments, Sales, Feedback)

Activity Network & Critical Path

Shows activities, estimated durations, and interdependencies. The **critical path** is the longest path — any delay on it delays the ENTIRE project.

Gantt Chart

Visual timeline with horizontal bars representing tasks. Shows: task names, start/end dates, durations, dependencies, overlapping parallel tasks, milestones, and deliverables.

4.5 Risk Management

A risk is an anticipated **unfavourable event** that can occur while a project is underway. Software risks are often intangible — syntax errors, third-party issues, team conflicts.

Risk Type	Definition	Examples	Mitigation Strategies
Technical	Related to technology, tools, 3rd-party components	• Wrong functions/UI built • 3rd-party component bug • Shortcomings in external APIs	• Communicate with clients, build prototypes • Benchmarking & regular inspections of 3rd-party components
Project	Budget, schedule, personnel, resources, customer issues	• Schedule slippage • Personnel shortfall • Insufficient domain knowledge	• Detailed milestones • Cross-training • Hire relevant experience • Frequent communication

Business	Harm business aspects of the product	• Product not competitive • Gold plating (unnecessary features)	• Explore market for similar products • Cost-benefit analysis • Communicate with clients
-----------------	--------------------------------------	---	--

Risk Assessment

$$\text{Risk Exposure} = \text{Probability} \times \text{Impact}$$

If given RE and Probability → Impact = RE / Probability (DON'T multiply!)

Risk	Probability	Impact	RE (P×I)
Schedule slippage	60%	3 (Critical)	1.8
Lack of experience	60%	3 (Critical)	1.8
Personnel shortfall	40%	3 (Critical)	1.2
DB scaling issues	20%	2 (Marginal)	0.4

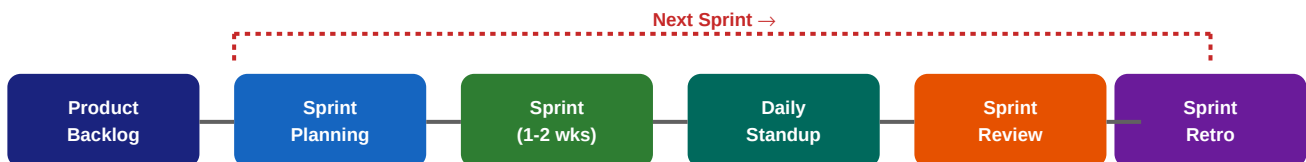
■ EXAM ALERT

Risk questions appear in EVERY paper:

- Third-party component bug = TECHNICAL risk (not project or business!)
- Mitigation for 3rd-party issues = 'Benchmarking and regular inspections' (NOT 'avoid 3rd-party altogether')
- 'Avoid third-party modules' is an EXTREME/unrealistic option — always wrong
- Risk Exposure = $P \times I$. If given RE and P, find Impact = RE/P (DON'T multiply!)
- Example: $RE=1.8, P=0.9 \rightarrow \text{Impact} = 1.8/0.9 = 2$

4.6 Agile Project Management — Scrum

In the Agile perspective, development is divided into short iterations. User stories are implemented in each iteration and prioritized for the next one. Team size: **4-9 people**.



Scrum Element	What It Is	Who Is Involved	Key Detail
Sprint	Short time-boxed period to complete work	Entire team	1-2 weeks duration
Product Backlog	Prioritized list of all work (from user stories)	Maintained by Product Owner	Items prioritized in sprint planning
Sprint Planning	Decide WHAT work + HOW it gets done	PO + SM + Dev team	2 hours per week of sprint
Daily Standup	Brief daily sync meeting	Dev team + SM + PO	3 Qs: Did what? Doing what? Blocked by?
Sprint Review	Team DEMONSTRATES completed work	Team + stakeholders	Shows working software (DEMO)
Sprint Retrospective	Team REFLECTS on process improvement	Team only	What went well? What didn't? How to improve?
Scrum Master	Ensures all Scrum activities done well	Facilitator role	NOT the project manager
Product Owner	Interfaces between client & dev team	Business representative	Prioritizes backlog

■ EXAM ALERT

Sprint Review vs Retrospective is ALWAYS tested:

- Sprint Review = 'What did we BUILD?' → Team DEMONSTRATES/demos completed work to stakeholders
- Sprint Retrospective = 'How can we IMPROVE?' → Team reflects on team dynamics, processes, tools
- Don't confuse them! Review = show work. Retro = reflect on process.

Velocity & Story Points

Story Points: Rate each user story's complexity — 1 (straightforward), 2 (medium), 3 (very complex).

Velocity = total number of story **POINTS** completed per sprint. NOT the number of stories!

■ EXAM ALERT

This is a guaranteed exam question:

- Team completed 2 stories totaling 5 points. Velocity = ?
- Answer: 5 (NOT 2, NOT 2.5, NOT 10)
- Velocity = total POINTS, not story count. Not average. Not product.

Plan-and-Document scheduling: WBS, Gantt charts, milestones

Agile scheduling: User stories, Points, Velocity

WEEK 5

Software Design

■ Topics Covered This Week

1. Design Process Outcomes: Components, Interfaces, Data Structures, Algorithms
2. Characteristics of Good Design: Correctness, Efficiency, Maintainability, Understandability
3. Design Modularity: HIGH Cohesion + LOW Coupling = Best Design
4. Types of Coupling (5 types from worst to best)
5. Types of Cohesion (5+ types from worst to best)
6. Object Oriented Design: Objects, Classes, Encapsulation, Relationships (4 types)
7. UML Diagrams: Structural (Class Diagram), Behavioral (Sequence Diagram, Activity, State Machine)
8. Software Architecture Patterns: P2P, Client-Server, MVC, Pipe & Filter
9. SOLID Principles (5 principles explained)
10. Design Patterns: Facade, Adapter, Builder, Strategy
11. Cyclomatic Complexity: $CC = \text{Decision Points} + 1$

5.1 Outcomes of the Design Process

The design phase produces four key outputs:

- **Components (Modules):** Collection of functions and data, each accomplishing a well-defined task
- **Interfaces:** How components communicate with each other — the 'contracts' between modules
- **Data Structures:** Suitable structures for storing and managing data within each component
- **Algorithms:** Logic required to implement each component's functionality

Components and Interfaces together form the **Software Architecture** (high-level design). Data structures and algorithms form the **detailed design**.

5.2 Characteristics of Good Software Design

Attribute	Definition	Example / Exam Scenario
Correctness	Correctly implements ALL functionalities of the system	All requirements satisfied without errors
Efficiency	Resources (time, space, cost) managed well	Mobile game restricts image resolution, reuses textures for low-end devices
Maintainability	Easy to change, modify, fix bugs, extend	Well-documented code, modular design
Understandability	Can be understood by everyone on the dev team	Clear naming, good documentation, simple architecture

■ EXAM ALERT

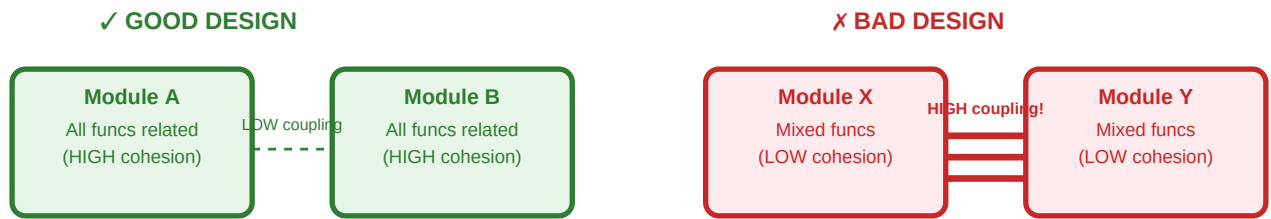
'Restricts image resolution + reuses textures for low-end devices' = EFFICIENCY

Not maintainability (that's about easy modification) or scalability (that's about handling growth).

5.3 Coupling and Cohesion — The Core of Modular Design

This is one of the **MOST TESTED** topics across both papers. The golden rule:

Goal: HIGH Cohesion + LOW Coupling = Best Modular Design



5.3.1 Types of Coupling (Worst → Best)

Coupling = measure of the degree of interaction/dependency between two modules. We want LOW coupling.

Type	Level	Description	How to Identify in Exam	Example
Content	WORST	Module accesses another's INTERNAL data/code directly	Rarely happens in Python/Java (language prevents it)	Module A directly reads Module B's private variables
Common	Bad	Two modules share GLOBAL data	Look for 'global variable', 'shared global data'	OPERATION_FLAG is a global variable both modules use
Control	Moderate	One module passes a FLAG/SWITCH to control another's behavior	Look for 'operationFlag', 'control variable', 'flag decides'	perform_actions(OPERATION_FLAG) where 0=View, 1=Add, 2=Delete
Stamp	Decent	Share composite data structure, each uses only part	Passing whole object when only part needed	Passing entire Customer object when only name is needed
Data	BEST	Communicate through simple, needed parameters only	Simple primitive types (int, float, string)	view_inventory(seller_id) — just passing an ID

■ EXAM ALERT

Coupling is tested in EVERY paper:

- 'operationFlag' or 'control variable' or 'flag that decides behavior' = CONTROL coupling
- 'Global variable shared between modules' = COMMON coupling
- 'Simple seller_id passed as parameter' = DATA coupling
- Memory trick: Content > Common > Control > Stamp > Data (worst to best)
- Mnemonic: 'Crazy Cats Chase Small Dogs'

5.3.2 Types of Cohesion (Worst → Best)

Cohesion = measure of how well functions within a module cooperate together for a single objective. We want HIGH cohesion.

Type	Level	Description	Key Identifier	Example
Coincidental	WORST	Functions grouped RANDOMLY, no meaningful relationship	No logical reason for grouping	login() + calculateTax() + printReport() in one module
Procedural	Poor	Functions executed in SEQUENCE but perform DIFFERENT tasks	'Executed one after another' + 'DISTINCT tasks'	UpdateStock() → GenerateReport() → SendNotification()
Communicational	Moderate	All functions operate on the SAME data/data structure	Functions use same data but do different things	Product module: add/edit/delete for both catalogue AND inventory
Sequential	Good	OUTPUT of one function is INPUT to the next	Data FLOWS from function to function	create_order() → output → check_availability() → output → place_order()

Functional	BEST	ALL functions contribute to a SINGLE well-defined task	All functions serve ONE purpose	Inventory module: view_inventory(), add_item(), delete_item() — all about inventory
-------------------	------	--	---------------------------------	--

■ EXAM ALERT

Sequential vs Procedural is the TRICKIEST distinction:

- SEQUENTIAL: output of one function FEEDS INTO the next (data flows)
- PROCEDURAL: functions just executed IN ORDER, but perform DIFFERENT tasks and output of one is NOT input to next

Key phrases for Procedural: 'loosely related' + 'distinct tasks' + 'executed one after another'

Key phrases for Sequential: 'output of one is input to next'

Also tested: Focused modules sharing a database = High cohesion + High coupling

5.4 Object Oriented Design

Object = key building block. Contains **Data** (attributes) + **Methods** (functions). **Encapsulation** = bundling data and methods together, ensuring data hiding/abstraction. Encapsulation ensures low coupling and high cohesion.

Class = template for creating objects. All objects with similar attributes and methods belong to a class. E.g., Electronics, Books, Clothing are all objects of type Product.

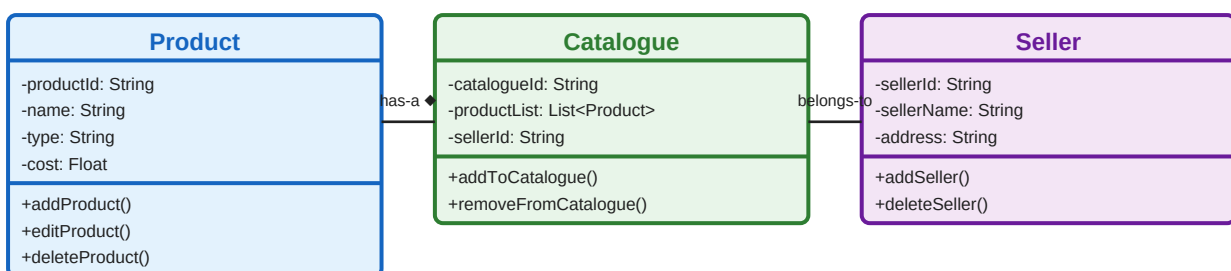
Class Relationships

Relationship	Definition	How to Identify (Keywords)	UML Notation	Example
Association	Classes take each other's help to perform functions	'A uses B', 'A calls method on B', 'A delegates to B'	Simple line →	Seller calls Inventory.viewInventory(sellerID)
Composition	Whole/part relationship (strong 'has-a')	'B is part of A', 'A contains B', 'A has a collection of Bs'	Filled diamond ◆→	Catalogue HAS a list of Products
Inheritance	Extend features of existing class ('is-a')	'A is a kind of B', 'A specializes B'	Triangle ▴→	Electronics, Books EXTEND Product
Dependency	Changes in A require changes in B	'B implements interface A'	Dashed arrow -.->	Inventory implements InventoryInterface

■ EXAM ALERT

Relationship questions: Seller and Inventory = ASSOCIATION (uses-a). Inventory and InventoryInterface = DEPENDENCY (implements). Seller and Inventory ≠ Inheritance (Seller doesn't extend Inventory).

5.5 UML Diagrams



View	Diagram Type	What It Shows	Key Question It Answers
------	--------------	---------------	-------------------------

Structural	Class Diagram	Classes, attributes, methods, relationships (static)	What are the components and how are they related?
Behavioral	State Machine	Different states an object goes through	What states can this object be in?
Behavioral	Activity Diagram	Flow of activities/actions (like a flowchart)	What is the workflow/process?
Behavioral	Interaction/Sequence	Sequence of MESSAGE EXCHANGES between objects	How do objects communicate over time?

■ EXAM ALERT

'Sequence of message exchanges among parts of a system' = INTERACTION diagram

Activity diagram = flow of ACTIVITIES/actions (NOT messages between objects)

Don't confuse these — Activity = workflow, Interaction/Sequence = message passing

Purposes of UML: Vehicle for communication and idea generation; Guide development; Generate code from models (close correspondence with implementation).

5.6 Architecture Patterns

Pattern	Description	Key Identifier	Example
Peer-to-Peer (P2P)	Every node is both client AND server; NO central server	'No central server' + 'users share directly'	BitTorrent, file sharing between users
Client-Server	Centralized server serves multiple clients	Central server handles all requests	Web applications, email
Pipe and Filter	Data flows through processing stages sequentially	Data transformed step by step	Unix command pipelines, compilers
MVC	Separates Model (data), View (UI), Controller (logic)	Separation of concerns in web apps	Django, Rails, most web frameworks

■ EXAM ALERT

P2P is in EVERY paper: 'No central server' + 'users download/upload directly from each other' +

'each user shares storage' = Peer-to-Peer. NOT client-server (which HAS a central server).

5.7 SOLID Principles

Prin ciple	Full Name	Rule	Violation Clue	Example
S	Single Responsibility	One class = ONE reason to change	Class doing too many unrelated things	User class handling auth AND email AND logging
O	Open-Closed	Open for EXTENSION, closed for MODIFICATION	Must modify existing code to add feature	Adding new shape requires editing Shape class
L	Liskov Substitution	Subtypes must be SUBSTITUTABLE for base types	Subclass breaks parent class behavior	Square extends Rectangle but breaks setWidth()
I	Interface Segregation	Don't FORCE clients to implement unused methods	'Forced to implement' methods not needed	SmartLight forced to implement play_music()
D	Dependency Inversion	Depend on ABSTRACTIONS, not concretions	High-level module directly depends on low-level	Payment depends on specific CreditCard instead of PaymentMethod interface

■ EXAM ALERT

ISP is in EVERY paper:

- 'DigitalOrderProcessor FORCED to implement package_order() and deliver_to_address()' = ISP violation
- 'SmartLight and SmartThermostat FORCED to implement play_music()' = ISP violation
- Keyword 'FORCED to implement unused methods' = ALWAYS Interface Segregation Principle
- Don't confuse with SRP (doing too many things) — ISP is about the INTERFACE being too fat

5.8 Design Patterns

Pattern	Problem It Solves	Key Identifier	Example
Facade	SIMPLIFIES a complex subsystem behind one easy interface	'Single simple method hiding many steps'	CoursePublisher.publish() hides verify+plagiarism+convert+catalog
Adapter	Makes INCOMPATIBLE interfaces work together	'Incompatible' interfaces need to connect	PaymentProcessor (USD) + gateways (EUR/INR) → CurrencyAdapter
Builder	Constructs complex objects STEP BY STEP	Building object incrementally with options	Building a complex meal: choose burger + drink + side + dessert
Strategy	Defines INTERCHANGEABLE algorithms at runtime	Switch between different algorithms	Different sorting algorithms; different compression methods

■ EXAM ALERT

Facade vs Adapter — KEY DIFFERENCE:

- Facade = SIMPLIFYING complexity (one simple method wrapping many complex steps)
- Adapter = making INCOMPATIBLE things work together (converting one interface to another)
- 'Single class with one simple method wrapping complex multi-step process' = FACADE
- 'PaymentProcessor only handles USD but gateways provide EUR/INR' = ADAPTER

5.9 Cyclomatic Complexity

A metric to measure code complexity. Formula:

CC = Number of Decision Points + 1

Decision points: if, for, while, elif, and, or (in conditions)

NOT a decision point: else (it's just the default path, not a decision)

Worked Example

Code: def classify_numbers(numbers):

- **for** num in numbers: ← Decision point 1
- **if** num > 0: ← Decision point 2
- classification['positive'].append(num)
- **else:** ← NOT a decision point
- **if** num < 0: ← Decision point 3

CC = 3 + 1 = 4

■ EXAM ALERT

Cyclomatic Complexity appears in EVERY paper:

- Count: if, for, while, elif, and, or → these are decision points
- DON'T count: else, return, function definition
- DON'T forget the +1 in the formula
- DON'T forget to count for/while loops
- A nested 'if' inside an 'else' IS still a decision point

Class Diagram Example – Seller Portal

Typical classes: **User**, **Product**, **Inventory**, **Order**, **Feedback**. Relationships: A **Seller** (subclass of **User**) *has* an **Inventory**; **Order** *contains* **Product** items.

7.4 Behavioural View: Sequence Diagram (Pages 60-62)

Shows interactions between objects over time using messages. Useful for describing a specific scenario.

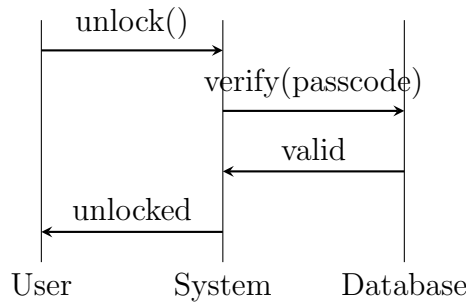


Figure 1: Sequence diagram for unlocking door (simplified)

7.5 Other Behavioural Views

- **State Machine View** – models different states of an object (e.g., **Locked**, **Unlocked**) and transitions.
- **Activity View** – flow of control among computational activities (similar to flowchart).
- **Interaction View** – sequence of message exchanges.

7.6 Purposes of UML Diagrams (Pages 65-69)

1. **Vehicle for communication and idea generation** – team members can discuss and refine designs visually.
2. **Guide development of software** – diagrams serve as blueprints for coding.
3. **Generate code from models** – forward engineering (some tools convert UML to code like Python/Java).
4. **Close correspondence with implementation** – ideally every class in the model maps to one or more classes in code.

8 VeriSIM: A Learning Environment for UML

8.1 Introduction (Page 71)

VeriSIM is a tool that allows verifying designs by simulating scenarios. It helps develop an integrated understanding of class diagrams and sequence diagrams.

8.2 Example: Automated Door Locking System (Page 72)

8.2.1 Requirements

1. If the passcode hasn't been set yet, the user can register and enter a required passcode.
2. When the user chooses the lock option and enters the correct passcode, the door should lock. If incorrect, door remains unlocked.
3. When the user chooses the unlock option and enters the correct passcode, the door should unlock. If incorrect, door remains locked.

8.3 Sequence Diagram for Unlocking (Page 73)

The diagram (described) shows interactions between the user, the door system, and a passcode validator. Steps:

1. User selects unlock option.
2. System prompts for passcode.
3. User enters passcode.
4. System checks against stored passcode.
5. If correct, door unlocks; else door remains locked.

8.4 Class Diagram for Door Lock System

Classes: Door, PasscodeManager, User. Relationships: Door has a PasscodeManager; User interacts with Door.

8.5 Design Tracing Strategy (Page 74)

A scenario: “When the door is initially locked and the user selects the unlock option and enters the correct passcode, the door unlocks.” We can construct a state diagram modelling this scenario:

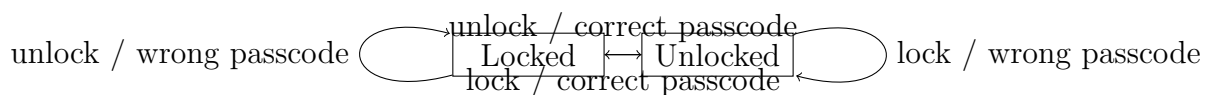


Figure 2: State diagram for door locking system

8.6 Activities in VeriSIM (Page 75)

Students can:

- Create UML diagrams
- Simulate scenarios to check consistency
- Trace design against requirements

BONUS

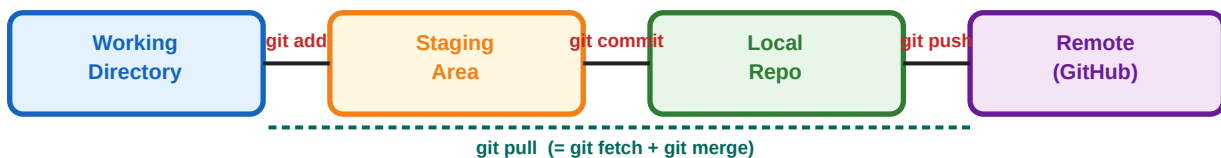
Version Control, Issue Tracking & More

B.1 Version Control Systems (VCS)

VCS helps track and manage changes to source code, maintain different versions, and track who changed what. Falls under **software configuration management**.

Feature	Centralized VCS	Distributed VCS (Git)
Repository	Single server repository everyone shares	Entire repo mirrored locally + remote
Dependency	Entirely dependent on server	Can work offline, push/pull when ready
If server dies	Development HALTS, may lose history	Every clone is a full backup
Speed	Network-dependent	Most operations LOCAL = fast
Examples	SVN, Perforce	Git, Mercurial

B.2 Git Workflow & Commands



File States in Git

- **Modified:** File changed in working directory (not yet staged)
- **Staged:** Marked to be committed — **git add filename**
- **Committed:** Changes stored in local database — **git commit -m 'message'**

Essential Git Commands

Command	What It Does	Key Detail
<code>git init</code>	Initialize new repository	Creates .git directory in current folder
<code>git clone url</code>	Mirror remote repository locally	Gets FULL history
<code>git add filename</code>	Stage modified/new files	Only staged files will be committed
<code>git commit -m 'msg'</code>	Commit staged files to local DB	Creates a snapshot/checkpoint
<code>git push</code>	Push local commits to remote	Sends your work to GitHub/remote
<code>git pull</code>	Fetch + merge from remote	= <code>git fetch</code> + <code>git merge</code> (auto-merges!)
<code>git fetch</code>	Download from remote, NO merge	Just downloads; you review first
<code>git status</code>	View file status	Shows modified/staged/untracked files
<code>git diff</code>	Show changes	Between commits, working tree, etc.
<code>git log</code>	View commit history	Shows all previous commits
<code>git branch name</code>	Create new branch	Branch = pointer to a commit
<code>git checkout branch</code>	Switch branches	Or use <code>git switch</code> (newer)
<code>git merge branch</code>	Merge branch into current branch	Combines two lines of development

git rebase master	Change base of current branch	Replay your commits on top of master
--------------------------	-------------------------------	--------------------------------------

■ EXAM ALERT

git pull vs git fetch appears in EVERY paper:

- git pull = fetch + merge (automatically merges into your branch)
- git fetch = download ONLY (does NOT auto-merge)
- Statement 'git pull fetches and automatically merges' = TRUE
- Statement 'git fetch retrieves and automatically merges' = FALSE

Remember: git pull = git fetch + git merge

Branching & Merging

Branches are independent versions of the repository. The default branch is 'master' (or 'main').

Typical workflow:

1. **git checkout -b feature1** — create and switch to new branch
2. Make changes, git add, git commit
3. **git checkout master** — switch back to master
4. **git merge feature1** — merge feature into master
5. **git branch -d feature1** — delete the feature branch

Git Rebase

When you need changes from master that were added AFTER you created your branch: **git checkout feature1** then **git rebase master**. This changes the base of your branch to the latest master commit, so you have access to the new code.

B.3 Issue Tracking & Code Review

- **Issue Tracking System:** Tool to manage and track all reported issues (GitHub Issues: create, assign, label, link to branches/PRs, close when fixed)
- **Code Review:** Peers examine code changes for quality, correctness, best practices, documentation. Done via Pull Requests.
- **Pair Programming:** Two programmers at one workstation — one writes code, one reviews simultaneously. Switch roles frequently.

Pull Request Workflow

1. Create branch for issue fix
2. Make changes, commit
3. Create Pull Request on GitHub
4. Peers review code ('Files changed' tab)
5. Approve changes
6. Merge to main branch
7. Issue automatically closed

B.4 Additional Topics

IDE vs Text Editor

An **IDE** (Integrated Development Environment) includes: code editor + compiler/interpreter + debugger + project management + auto-completion. Examples: VS Code, PyCharm, Thonny.

A **text editor** is a simple tool for editing text. No built-in compiler, debugger, or project management. Example: **TextEdit (Mac)**, Notepad, nano.

■ EXAM ALERT

'Which is NOT an IDE?' → TextEdit (Mac) is NOT an IDE. Thonny IS an IDE (beginner Python IDE with debugger). VS Code IS an IDE. PyCharm IS an IDE.

Debugging Techniques

Technique	When to Use	Key Identifier
Black Box	Can't see internal code (3rd-party library)	'Third-party' + 'reviews documentation/examples'
Forward	Trace execution from start to find error	Step through code line by line from beginning
Backward	Start from error, trace back to cause	Start at the bug and work backwards
Input Manipulation	Vary inputs to isolate the bug	Systematically changing inputs to find trigger

■ EXAM ALERT

Black box debugging appears in BOTH papers:

- 'Third-party' + 'cannot see internal code' + 'reviews API documentation and examples' = BLACK BOX
- Key: you can't look inside, so you debug from outside using docs and input/output

EXAM PREP

Practice MCQs & Exam Tips

Practice MCQs — Set A (Based on Paper 1)

Q1. Faculty dashboard to manage courses = Internal, Library syncing with archive = Another Software, Online portal for prospective students = External. Match:

- (a) A-1, B-2, C-3
- (b) A-3, B-1, C-2
- (c) A-2, B-3, C-1
- (d) A-1, B-3, C-2

✓ **Answer: (b) A-3, B-1, C-2** — External=prospective students, Internal=faculty, Software=library sync

Q2. Team studies Paytm and PhonePe before building digital wallet. Activity?

- (a) Software Testing
- (b) Requirements Analysis
- (c) Code Optimization
- (d) Alpha Testing

✓ **Answer: (b) Requirements Analysis** — Studying similar apps = understanding what to build

Q3. Professors=Primary, Students=Secondary, Accreditation body=?

- (a) Primary
- (b) Secondary
- (c) Tertiary
- (d) External

✓ **Answer: (c) Tertiary** — 'Occasionally reviews' = Tertiary

Q4. Password every 30 days vs no changes more than once/year. Issue?

- (a) Incomplete
- (b) Ambiguous
- (c) Inconsistent
- (d) Implicit

✓ **Answer: (c) Inconsistent** — Both clear individually but directly contradict

Q5. Client wants email alerts during implementation. Agile principle?

- (a) Working software over docs
- (b) Customer collaboration over contracts
- (c) Processes over individuals
- (d) Plan over change

✓ **Answer: (b) Customer collaboration over contracts** — Options c,d are REVERSED Agile values!

Q6. Reading banking policies and competitor FAQs. Technique?

- (a) Focus group
- (b) Observation
- (c) Documentation
- (d) Interview

✓ **Answer: (c) Documentation** — Reading existing written materials = Documentation

Q7. User story lacks deadline. Missing SMART criterion?

- (a) Specific
- (b) Measurable
- (c) Achievable

(d) Time-boxed

✓ Answer: (d) Time-boxed — No deadline = T in SMART

Q8. Clickable PowerPoint screens, users try placing order. Prototype?

- (a) Storyboard
- (b) Paper prototype
- (c) Digital mock-up
- (d) Interactive prototype

✓ Answer: (d) Interactive prototype — 'Clickable' + navigation = Interactive

Q9. Scrum: 2 stories totaling 5 points. Velocity?

- (a) 2
- (b) 5
- (c) 2.5
- (d) 10

✓ Answer: (b) 5 — Velocity = total POINTS (not story count!)

Q10. App no real-time delivery tracking. Heuristic violated?

- (a) Freedom
- (b) Flexibility
- (c) Recognition over recall
- (d) Show Status

✓ Answer: (d) Show Status — No progress updates = Show Status

Q11. Mobile game: restrict resolution, reuse textures. Quality?

- (a) Understandability
- (b) Maintainability
- (c) Efficiency
- (d) Scalability

✓ Answer: (c) Efficiency — Optimizing resources = Efficiency

Q12. Can't see 3rd-party library code. Debug using docs. Technique?

- (a) Step into using PDB
- (b) Forward debugging
- (c) Black box debugging
- (d) Dump and diff

✓ Answer: (c) Black box debugging — Can't see inside + use docs = Black box

Q13. for loop + if/else inside + if after. Cyclomatic complexity?

- (a) 2
- (b) 3
- (c) 4
- (d) 5

✓ Answer: (c) 4 — for(1)+if(1)+if(1)=3 decisions. CC=3+1=4. else NOT counted!

Q14. File sharing, no central server, each user shares storage. Architecture?

- (a) Client-Server
- (b) MVC
- (c) Pipe and Filter
- (d) Peer-to-Peer

✓ Answer: (d) Peer-to-Peer — No central server + share directly = P2P

Q15. DigitalOrderProcessor forced to implement package_order(). SOLID?

- (a) Single Responsibility
- (b) Interface Segregation
- (c) Open-Closed

(d) Liskov

✓ Answer: (b) Interface Segregation — 'Forced to implement unused methods' = ISP

Q16. One simple method hides verify+plagiarism+convert+catalog. Pattern?

- (a) Builder
- (b) Adapter
- (c) Facade
- (d) Strategy

✓ Answer: (c) Facade — Simple interface hiding complex subsystem = Facade

Q17. 12-month strict sequential, client unhappy at end. Model?

- (a) Agile
- (b) Spiral
- (c) Waterfall
- (d) Prototype

✓ Answer: (c) Waterfall — Strict sequential + late feedback = Waterfall

Q18. SellerModule calls with operationFlag (0=View,1=Add,2=Delete). Coupling?

- (a) Data
- (b) Content
- (c) Control
- (d) Common

✓ Answer: (c) Control — Flag controlling behavior = Control coupling

Practice MCQs — Set B (Based on Paper 2)

Q19. TextEdit (Mac), VS Code, PyCharm, Thonny — which is NOT an IDE?

- (a) VS Code
- (b) PyCharm
- (c) Thonny
- (d) TextEdit

✓ Answer: (d) TextEdit — Basic text editor, no debugger/compiler

Q20. git pull fetches and auto-merges (S1). git fetch auto-merges (S2). Which correct?

- (a) Both correct
- (b) Only S1 correct
- (c) Only S2 correct
- (d) Both wrong

✓ Answer: (b) Only S1 correct — git fetch does NOT auto-merge

Q21. 3rd-party payment API, reviews documentation. Debugging strategy?

- (a) Input manipulation
- (b) Backward
- (c) Forward
- (d) Black box

✓ Answer: (d) Black box — 3rd-party + documentation = Black box

Q22. Open-ended, no predefined Qs, adapts to responses. Interview type?

- (a) Structured
- (b) Semi-structured
- (c) Unstructured
- (d) Questionnaire

✓ Answer: (c) Unstructured — 'No predefined set' = Unstructured

Q23. UpdateStock(), GenerateReport(), SendNotification() - in order, different tasks. Cohesion?

- (a) Coincidental
- (b) Procedural
- (c) Communicational
- (d) Sequential

✓ Answer: (b) Procedural — In order + distinct tasks = Procedural

Q24. Focused modules sharing a database. Statement?

- (a) High cohesion, high coupling
- (b) Low cohesion, high coupling
- (c) High cohesion, low coupling
- (d) Low cohesion, low coupling

✓ Answer: (a) High cohesion, high coupling — Focused = high cohesion. Shared DB = high coupling

Q25. Message exchanges among parts of system. Which diagram?

- (a) Class diagram
- (b) State machine
- (c) Activity diagram
- (d) Interaction diagram

✓ Answer: (d) Interaction diagram — Message exchanges = Interaction (sequence) diagram

Q26. Sprint retrospective objective?

- (a) Decide sprint work
- (b) Prioritize backlog
- (c) Demo completed work
- (d) Evaluate team dynamics/processes

✓ Answer: (d) Evaluate team dynamics/processes — Retro = reflect on process. Review = demo.

Q27. RE for budget slippage = 1.8, probability = 0.9. Impact?

- (a) 1.62
- (b) 2
- (c) 3
- (d) 0.17

✓ Answer: (b) 2 — Impact = RE/Probability = $1.8/0.9 = 2$

Q28. Bug found after deployment by customers. SDLC phase?

- (a) Requirements
- (b) Testing
- (c) Development
- (d) Maintenance

✓ Answer: (d) Maintenance — Bug AFTER deployment = Maintenance

Q29. User story too big for one sprint, subdivided. SMART criterion achieved?

- (a) Specific
- (b) Measurable
- (c) Achievable
- (d) Time-boxed

✓ Answer: (c) Achievable — Too big → break down → now Achievable

Q30. PaymentProcessor (USD) vs gateways (EUR, INR). Design pattern?

- (a) Factory
- (b) Builder
- (c) Adapter
- (d) Facade

✓ Answer: (c) Adapter — Incompatible interfaces = Adapter

Q31. SmartLight forced to implement play_music(). SOLID principle?

- (a) Open-Closed
- (b) Liskov
- (c) Interface Segregation
- (d) Dependency Inversion

✓ **Answer: (c) Interface Segregation — Forced unused methods = ISP**

Q32. R1: 500 logins. R2: View doctors. R3: Notify booking. R4: Process in 3s. Functional?

- (a) R1,R4 functional
- (b) R2,R3 functional; R1,R4 non-functional
- (c) All functional
- (d) All non-functional

✓ **Answer: (b) R2,R3 functional; R1,R4 non-functional — Features=Functional; Performance numbers=Non-functional**

Q33. classify_numbers: for + if + if inside else. CC?

- (a) 3
- (b) 4
- (c) 5
- (d) 6

✓ **Answer: (b) 4 — for(1)+if(1)+if(1)=3. CC=3+1=4. Don't count else!**

Q34. Government traffic control, hardware integration, real-time, 100 KLOC. COCOMO type?

- (a) Organic
- (b) Semi-detached
- (c) Embedded
- (d) Cannot determine

✓ **Answer: (c) Embedded — HW integration + real-time + govt = Embedded**

Q35. Complete product, real-world scenarios, actual users, before final release. Testing?

- (a) System testing
- (b) Integration testing
- (c) Alpha testing
- (d) Beta testing

✓ **Answer: (d) Beta testing — Actual users + real-world + before release = Beta**

CHEAT SHEET

Quick Revision Before the Exam

SDLC & Models

Phases: Requirements → Design → Development → Testing → Maintenance

Waterfall: Sequential, no going back, client sees product at END, 6-18 months

Prototype: Build throwaway prototype first, get feedback, then build real system

Spiral: Iterative cycles with emphasis on RISK ANALYSIS in each cycle

Bug after deployment = Maintenance | Bug before release = Testing

Agile & Scrum

4 Values: Individuals > Processes | Working SW > Docs | Customer Collab > Contracts | Change > Plan

WATCH FOR REVERSED VALUES IN EXAM OPTIONS!

Sprint Planning(what+how) → Daily Standup(3Qs) → Sprint Review(DEMO) → Retrospective(REFLECT)

Review = 'What did we build?' | Retrospective = 'How do we improve?'

Velocity = total story POINTS per sprint (NOT story count, NOT average)

Requirements

Primary(frequent,central) | Secondary(consumers) | Tertiary(occasional/indirect)

Techniques: Questionnaires | Interviews(Struct/Semi/Unstruct) | Focus Groups | Observation | Documentation

Unstructured interview: NO predefined questions, fully adaptive

Functional = WHAT it does | Non-functional = HOW WELL (numbers = NFR)

SMART: Specific, Measurable, Achievable, Relevant, Time-boxed

Inconsistent = two requirements that directly CONTRADICT each other

UI & Heuristics

Usability: Effectiveness | Efficiency | Safety | Learnability | Memorability

Learnability features: wizards, templates, tooltips (NOT keyboard shortcuts = efficiency)

Prototypes: Storyboard → Paper → Digital Mock-up → Interactive ('clickable' = key!)

UNDERSTANDING: Consistency, Familiar Metaphors, Clean Design

ACTION: Freedom(undo), Flexibility(shortcuts), Recognition>Recall

FEEDBACK: Show Status, Prevent Errors, Error Recovery, Provide Help

No delivery tracking = Show Status | 'Did you mean?' = Error Recovery

COCOMO & Estimation

Effort = $a \times (KLOC)^b$ | Organic(2.4,1.05) | Semi-detached(3.0,1.12) | Embedded(3.6,1.20)

Step 1: Determine type → Step 2: Initial effort → Step 3: EAF = PRODUCT of multipliers → Step 4: Final = Initial × EAF

Indian notation: 1,00,000 = 100,000 = 100 KLOC

Risk Exposure = Probability × Impact | Impact = RE / Probability

3rd-party bug = Technical risk | Mitigation = Benchmarking + regular inspections

Delphi: Multiple experts → independent estimates → coordinator summarizes → iterate to consensus

Design & OOP

HIGH Cohesion + LOW Coupling = BEST design

Coupling (worst→best): Content > Common > Control > Stamp > Data

Flag/switch = Control | Global variable = Common | Simple parameter = Data

Cohesion (worst→best): Coincidental > Procedural > Communicational > Sequential > Functional
Sequential: output feeds into next | Procedural: just executed in order, different tasks
Shared database between focused modules = High cohesion + High coupling
SOLID: S(one reason) O(extend not modify) L(substitutable) I(no forced methods) D(abstractions)
'Forced to implement unused methods' = ALWAYS ISP (Interface Segregation)
Facade = simplify complexity | Adapter = incompatible interfaces | CC = decisions + 1 (else NOT counted)

Git & Version Control

git pull = fetch + merge (auto) | git fetch = download only (NO merge)
File states: Modified → Staged (git add) → Committed (git commit)
Branch: git branch name → git checkout name → make changes → git merge back
Rebase: change base of branch to latest master (git rebase master)
P2P = no central server, each node is client+server | Centralized VCS depends on server
Black box debugging: can't see internals, use documentation + examples
TextEdit (Mac) = NOT an IDE | Thonny = IS an IDE
Alpha = internal testers | Beta = external actual users in real-world

■ Top Exam Strategy

- **1. Read EVERY word carefully.** 'Occasionally', 'forced', 'strict sequential', 'no central server', 'clickable' — these are deliberate clues.
- **2. Watch for REVERSED Agile values** in options — they flip the 'over' to trick you.
- **3. COCOMO: Always determine project type FIRST** from the description, then pick constants.
- **4. Coupling keywords:** Flag/switch = Control | Global = Common | Simple parameter = Data.
- **5. Cohesion:** Output feeds into next = Sequential | Executed in order, different tasks = Procedural.
- **6. 'Forced to implement unused methods'** = Interface Segregation Principle. ALWAYS.
- **7. Facade = simplifying.** Adapter = incompatible. Different problems!
- **8. CC = decisions + 1.** Count if/for/while/elif. NEVER count else.
- **9. Alpha = internal.** Beta = external actual users. Bug after deployment = Maintenance.
- **10. git pull = fetch + merge.** git fetch = download only. This is guaranteed marks.
- **11. Risk formula:** $RE = P \times I$. If asked for Impact, divide (don't multiply!).
- **12. 'Third-party' + 'can't see code'** = Black box debugging. Every time.

Made by Anmol Kansal with AI help

Software Engineering — IIT Madras BS Degree Programme

■ **Best of luck with your exam! You've got this!** ■

SOFTWARE ENGINEERING

Week 7 & Week 8 — Detailed Notes

IIT Madras BS Degree Programme

- Week 7: Software Development — IDEs, Frameworks, Debugging, Software Metrics, Code Smells, Refactoring
 - Week 8: Software Architecture — MVC, P2P, SOLID Principles (Deep Dive), Design Patterns (6 patterns)
 - Every Topic Explained with Examples & Diagrams
 - 20+ Practice MCQs Based on Previous Year Papers
 - Exam Alerts, Tips & Tricks Throughout
 - Quick Revision Cheat Sheet
-

Made by Anmol Kansal with AI help

Dr. Sridhar Iyer (IIT Bombay) & Dr. Prajish Prasad (FLAME University)

WEEK 7

Software Development

■ Topics Covered This Week

1. Development Environment: Text Editors vs IDEs
2. Web Application Frameworks (Flask example)
3. Debugging: Definitions (Error, Failure, Fault), Steps, Techniques & Strategies
4. Python Debugger (PDB) commands
5. Software Metrics: Cyclomatic Complexity (deep dive), Raw Metrics (LOC, LLOC, SLOC), Halstead's Metrics
6. Code Smells: Comments, Functions, General (DRY principle)
7. Refactoring: Improving structure without changing behaviour

7.1 Development Environment

When you write software, you interact with code using a combination of tools. At the most basic level, you need a **text editor** to write code and a **compiler/interpreter** to run it. For example, you could use Notepad + Python runtime. However, professional development uses more sophisticated tools.

7.1.1 Text Editors vs IDEs

Feature	Text Editor	IDE (Integrated Development Environment)
What it is	Simple tool for editing plain text	Full-featured software development application
Syntax highlighting	Basic or none	Advanced, language-aware
Compiler/Interpreter	NOT included	Built-in or integrated
Debugger	NOT included	Built-in debugger with breakpoints, stepping
Project management	NOT included	File trees, project configuration, build tools
Auto-completion	Basic or none	Intelligent code completion (IntelliSense)
Refactoring tools	NOT included	Rename variables across codebase, extract methods
Examples	Notepad, TextEdit (Mac), Vim, Emacs, nano	VS Code, PyCharm, Thonny, IntelliJ IDEA

IDE provides additional features that make development more effective: syntax checking, finding where a function is used, renaming a variable/class throughout the entire codebase, integrated debugging, and version control integration.

EXAM ALERT

'Which is NOT an IDE?' → TextEdit (Mac) is NOT an IDE (it's a basic text editor).

Thonny IS an IDE (beginner-friendly Python IDE with built-in debugger).

VS Code IS an IDE (feature-rich with extensions for debugging, Git, IntelliSense).

PyCharm IS an IDE (full-featured Python IDE by JetBrains).

Key difference: IDE = editor + compiler + debugger + project management. Text editor = just text editing.

7.1.2 Web Application Frameworks

A **framework** is a collection of common features bundled together, built and maintained by a development community. It helps structure projects so developers can quickly build and maintain applications instead of writing everything from

scratch.

Flask is a Python web application framework that helps developers quickly start developing web apps. A basic Flask app:

- **from flask import Flask** — import the framework
- **app = Flask(__name__)** — create the app
- **@app.route('/')** — define a URL route
- **Templates:** Use `{{ variable }}` syntax to inject dynamic data into HTML pages

REMEMBER

Frameworks provide structure and common functionality so you don't reinvent the wheel. Flask is the example used in this course — it handles routing, templates, and web server setup.

7.2 Debugging

Debugging is one of the most important skills in software development. Before diving into techniques, let's understand the key terminology:

Term	Definition	Example
Error	Discrepancy between actual behaviour and intended behaviour	Code calculates wrong result
Failure	OBSERVABLE error — incorrect output, exception, crash	Program outputs 42 instead of expected 56
Fault	WHERE the failure has occurred (specific lines in code)	The bug is on line 15: used + instead of *
Debugging	The process of determining the CAUSE of a failure	Finding and fixing the root cause

ANALOGY

Think of it as a medical diagnosis: the patient has SYMPTOMS (failure — observable problems), the DISEASE (fault — the actual bug in the code), and the doctor's job is DIAGNOSIS (debugging — finding the root cause). The original mistake that caused everything is the ERROR.

7.2.1 Steps in Debugging

- **1. Reproduce the problem** — make the bug happen consistently
- **2. Find cause of defect** — locate WHERE in the code the bug is
- **3. Investigate fix** — determine the best way to fix it without breaking other things
- **4. Implement fix** — write the actual fix
- **5. Test fix** — verify the fix works and doesn't introduce new bugs

7.2.2 Debugging Techniques

Technique	How It Works	When to Use
Logging / Print Statements	Insert <code>print()</code> statements to see variable values and execution flow	Quick and simple debugging
Dump & Diff	Use diff tool to compare logging data between correct and incorrect executions	When you have a working version to compare against
Stepping in Debugger	Use PDB or IDE debugger to step through code line by line	When you need precise control flow analysis
Profiling Tool	Shows how often and how long various parts of program execute	When debugging performance issues

Research shows the top resources used in debugging are: **Communication with others** (14), **Internet resources** (10), **Custom code/manual debugging** (7), followed by code browsers, documentation, and monitoring systems.

7.2.3 Debugging Strategies

These are the **high-level approaches** to finding bugs — heavily tested in exams:

Strategy	How It Works	Key Exam Identifier	Example
Input Manipulation	Edit inputs, observe differences in output; systematically vary inputs to isolate the bug	'Vary inputs to find which triggers the bug'	Testing <code>first_n_primes</code> : input <code>2→[2,3]</code> , <code>3→[2,3]</code> — found the bug with odd inputs
Backwards	Start from incorrect OUTPUT, follow data and control dependencies BACKWARDS to find the faulty line	'Start from the error, trace back'	Wrong output → trace which variable is wrong → trace where it was set
Forwards	Find event that triggered incorrect behavior, follow control flow FORWARD until incorrect state reached	'Start from beginning, trace forward'	Step through from function entry, watching state change
Black Box	Can't see internal code; use documentation, examples, and input/output to debug	'Third-party' + 'can't see code' + 'documentation'	Third-party API bug — read API docs, check examples, test inputs

EXAM ALERT

Debugging strategy appears in BOTH previous papers!

- 'Third-party API' + 'cannot see internal code' + 'reviews documentation and examples' = BLACK BOX debugging
- 'Third-party payment service' + 'studying API documentation' = BLACK BOX
- Key: if you CAN'T look inside the code → Black Box
- If stepping through line by line → Forwards (using PDB)
- If starting from wrong output and tracing back → Backwards

7.2.4 Python Debugger (PDB)

PDB is Python's built-in debugger. To use it, add `import pdb; pdb.set_trace()` at the point where you want to start debugging.

PDB Command	Full Name	What It Does
n	next	Execute next line (step OVER function calls)
s	step	Step INTO function calls (go inside the function)
p expression	print	Print the value of an expression
b	break	Set a breakpoint
c	continue	Continue execution until next breakpoint

7.3 Software Metrics

Software metrics provide a **quantitative way to measure the quality** of your code. Why write clean code? Because it's easier to understand, test, maintain, and extend. Tools like **Radon** can analyse code automatically.

7.3.1 Cyclomatic Complexity — Deep Dive

Cyclomatic Complexity (CC) measures the number of **independent paths** through your code. Formula:

CC = Number of Decision Points + 1

Construct	CC Effect	Reasoning
if	+1	An if statement is a single decision
elif	+1	Each elif adds another decision

else	+0	else does NOT cause a new decision — the decision is at the if
for	+1	There is a decision at the start of the loop (enter or skip)
while	+1	There is a decision at the while statement
except	+1	Each except branch adds a new conditional path
and / or	+1 each	Each boolean operator in a condition adds a decision

CC Score interpretation:

CC Score	Rank	Risk Level
1-5	A	Low — simple block
6-10	B	Low — well structured and stable
11-20	C	Moderate — slightly complex
21-30	D	More than moderate — more complex
31-40	E	High — complex, alarming
41+	F	Very high — error-prone, unstable

EXAM ALERT

Cyclomatic Complexity is in EVERY exam paper! Common traps:

- else is NOT a decision point (CC +0)
- except IS a decision point (CC +1)
- for and while ARE decision points (CC +1 each)
- Don't forget the +1 at the end of the formula
- A nested if inside an else IS still counted

Worked Example: for + if + if inside else:

for(+1) + if(+1) + if_nested(+1) = 3 decisions → CC = 3+1 = 4

7.3.2 Raw Metrics

Metric	Full Name	What It Measures
LOC	Lines of Code	Total number of lines (including blanks, comments)
LLOC	Logical Lines of Code	Lines containing exactly ONE statement (no blanks, no comments)
SLOC	Source Lines of Code	Source lines of code — may not exactly match LLOC
Comments	Comment Lines	Number of comment lines in the code

7.3.3 Halstead's Metrics

Halstead's metrics measure code complexity based on **operators and operands**:

- **n1** = number of DISTINCT operators | **n2** = number of DISTINCT operands
- **N1** = TOTAL number of operators | **N2** = TOTAL number of operands
- **Program vocabulary:** $n = n1 + n2$
- **Program length:** $N = N1 + N2$
- **Volume:** $V = N \times \log_2(n)$
- **Difficulty:** $D = (n1/2) \times (N2/n2)$
- **Effort:** $E = D \times V$
- **Time to program:** $T = E / 18 \text{ seconds}$
- **Estimated bugs:** $B = V / 3000$

REMEMBER

You probably won't need to calculate Halstead's from scratch in the exam, but know what the metrics represent — especially Volume (size of implementation), Difficulty (how hard to understand), and Effort (mental effort to develop).

7.4 Code Smells & Clean Code

Code smells are **problematic characteristics** in code that suggest deeper issues. The term comes from Robert Martin's book 'Clean Code' which lists 63 code smells. Key categories:

7.4.1 Comment Smells

- **Redundant comments:** `i++ // Increment i by 1` — the code already says this!
- **Commented-out code:** Old code left as comments instead of being deleted (use version control instead)

7.4.2 Function Smells

- **Functions doing more than one thing:** Each function should do ONE thing only (relates to Single Responsibility)
- **Functions that are too long:** Hard to understand, test, and maintain
- **Too many arguments:** Functions with 5+ parameters are hard to call correctly
- **Flag arguments:** Passing booleans to control function behavior (leads to Control Coupling)
- **Dead functions:** Functions that are never called — remove them

7.4.3 General Smells

DRY — Don't Repeat Yourself

One of the most important principles. Clumps of identical code throughout the program should be extracted into simple methods.

Bad (violates DRY): `Checking month == 'April' or month == 'June' or month == 'September' or month == 'November'` repeated in multiple places.

Good (follows DRY): `thirty_day_months = ['April', 'June', 'September', 'November']` and check if month in `thirty_day_months`.

- **Incorrect behaviour at boundaries:** Not testing edge cases (empty lists, zero values, max values)
- **Use explanatory variables:** Instead of magic numbers, use named constants

7.4.4 Refactoring

Refactoring = changing the code by **improving its structure WITHOUT changing its behaviour**. The external functionality stays exactly the same, but the internal code quality improves.

Refactoring tools: Pylint, flake8, vulture (finds dead code)

EXAM ALERT

Refactoring = improving structure, NOT changing behavior. If behavior changes, it's not refactoring — it's a feature change or bug fix.

Code smells = warning signs that refactoring is needed.

DRY principle = avoid duplicated code by extracting into methods.

Flag arguments in functions → leads to Control Coupling.

WEEK 8

Software Architecture

■ Topics Covered This Week

1. Software Architecture: Definition, Components, Connectors, Protocols
2. Architecture Styles: Client-Server, Pipe & Filter, MVC (detailed), Peer-to-Peer
3. Design Patterns Introduction: Problem → Solution → Consequences
4. SOLID Principles Deep Dive (with class diagram examples for each)
5. Creational Patterns: Factory, Builder
6. Structural Patterns: Facade, Adapter
7. Behavioural Patterns: Iterator, Observer, Strategy

8.1 Software Architecture

Software architecture is the **way of organizing your code** — it defines software elements/modules, relations among them, and properties of both elements and relations.

Key Concepts

- **Component:** A well-defined functionality or behavior separate from other functionality. E.g., in Client-Server: the Client software and the Server are components.
- **Connector:** Code that transmits information between components; responsible for regulating interactions.
- **Protocol:** Set of pre-defined rules describing how components should interact. E.g., function calls, REST APIs, HTTP.

8.1.1 Architecture Styles

Style	How It Works	Data Flow	Key Identifier	Example
Client-Server	Centralized server responds to client requests	Request → Server → Response	Central server serves multiple clients	Web apps, email, databases
Pipe & Filter	Data passes component to component, transformed along the way	Input → Filter1 → Filter2 → Output	Data TRANSFORMED step by step	Unix pipes, compilers, image processing
MVC	Separates data (Model), presentation (View), logic (Controller)	User → Controller → Model → View	Separation of data, UI, and logic	Django, Rails, most web frameworks
Peer-to-Peer	Distributed — nodes share resources directly, NO central system	Node ↔ Node (bidirectional)	'No centralized system' + 'share directly'	BitTorrent, blockchain, file sharing

8.1.2 Model-View-Controller (MVC) — Detailed

MVC separates the data from its visual representation and the logic that coordinates them:

Component	Responsibility	Amazon Seller Portal Example
Model	Models the DATA required for the service; classes/objects that map to database tables	Product class, Seller class, Inventory class — model the database

View	The GUI/presentation layer; VISUAL REPRESENTATION of the Model	HTML pages showing product listings, seller dashboard
Controller	Coordinates multiple views; helps users MANIPULATE the model	Routes and handlers that process form submissions, button clicks

EXAM ALERT

MVC is tested alongside P2P:

- 'Views of data separated from manipulations of data' = MVC
- 'No centralized system' + 'nodes share directly' = P2P
- MVC is about CODE ORGANIZATION within one system
- P2P is about NETWORK ARCHITECTURE between systems

8.2 SOLID Principles — Deep Dive

SOLID principles are object-oriented design guidelines conceptualized by **Robert Martin (Uncle Bob)**. They make code easy to understand and modular. This week goes much deeper with class diagram examples for each principle.

S — Single Responsibility Principle (SRP)

Rule: Every class should have a **single responsibility/purpose** — only one reason to change.

Violation Example: A single Seller class that handles seller info AND product management AND inventory management. This class has THREE reasons to change.

Fix: Split into three classes — Seller (manages seller data), Product (manages product data), Inventory (manages inventory). Each has exactly ONE responsibility.

EXAM ALERT

SRP violation = class doing TOO MANY things (multiple responsibilities).

ISP violation = interface FORCING unused methods.

Don't confuse them: SRP = the class itself does too much. ISP = the interface demands too much.

O — Open-Closed Principle (OCP)

Rule: Software entities should be **open for extension** but **closed for modification**.

Violation Example: Adding a new product type 'Clothing' requires modifying the existing Product class to add size and colour fields and a new constructor.

Fix: Use inheritance — create a Clothing subclass that extends Product, adding size and colour without modifying the original Product class.

REMEMBER

Open for extension = you CAN add new functionality through subclasses.

Closed for modification = you DON'T change existing, tested code.

L — Liskov Substitution Principle (LSP)

Rule: Derived classes should be **substitutable** for their base class. If class D is a subtype of class B, you should be able to replace B with D without breaking the program.

Example from slides: FeedbackNotification is an abstract class with notify(Product, feedback, email). When adding SMS notifications, the SMS subclass is forced to accept an 'email' parameter it doesn't use.

Fix: Change abstract method to notify(Product, feedback) — each subclass stores its own contact info (EmailNotification has email field, SMSNotification has mobileNo field). Now either can substitute for the base class.

I — Interface Segregation Principle (ISP)

Rule: Do NOT force any client to implement an interface which is **irrelevant to them**.

Violation Example: DeliveryProcessing interface has placeOrder(), notifySeller(), packageOrder(), deliver(). But ONLINE orders don't need packageOrder() and deliver() — they need provideDownloadLink() instead.

Fix: Split into two interfaces — PhysicalDeliveryProcessing (with packageOrder + deliver) and OnlineDeliveryProcessing (with provideDownloadLink). Each class implements only what it needs.

EXAM ALERT

ISP is the MOST TESTED SOLID principle (appears in every paper):

- 'DigitalOrderProcessor FORCED to implement package_order()' = ISP violation
- 'SmartLight FORCED to implement play_music()' = ISP violation
- The keyword 'FORCED to implement methods it doesn't need' = ALWAYS ISP
- Fix: Split the 'fat' interface into smaller, focused interfaces

D — Dependency Inversion Principle (DIP)

Rule: Prefer **abstractions/interfaces** over concrete implementations. High-level modules should not depend on low-level modules — both should depend on abstractions.

Violation Example: Seller class directly depends on BankAccount class. What if seller wants payment via PayTM, Google Pay, or UPI?

Fix: Create a PaymentAccount interface with a payment() method. BankAccount, UPI, GooglePay, PayTM all implement this interface. Seller depends on PaymentAccount (abstraction), not on any specific implementation.

TIP

DIP = depend on ABSTRACTIONS (interfaces), not on specific classes. This makes it easy to swap implementations without changing the high-level code. Example: Seller works with any PaymentAccount, not just BankAccount.

8.3 Design Patterns

Design patterns are **descriptions of communicating objects and classes** customized to solve a general design problem in a particular context. They are NOT code — they are templates/blueprints.

Every design pattern has three elements:

- **Problem:** When and in what context to apply the pattern
- **Solution:** Elements, relationships, responsibilities, collaborations
- **Consequences:** Results, trade-offs, cost-benefit analysis

Design Smells = warning signs that your code may be heading towards an anti-pattern. SOLID guidelines help avoid design smells. **Refactoring** = restructuring code (moving code between classes, creating/removing classes) to fix design smells.

Category	Purpose	Patterns Covered
Creational	Used during the process of OBJECT CREATION	Factory, Builder
Structural	COMPOSITION of classes or objects	Facade, Adapter
Behavioural	Ways classes/objects INTERACT and distribute responsibility	Iterator, Observer, Strategy

8.3.1 Creational Patterns

Factory Design Pattern

Problem: You need to create objects, but the exact type depends on context. Using 'new' directly couples your code to specific classes.

Solution: Replace direct object construction with calls to a special **factory method**. The factory decides which subclass to instantiate.

Example: A Delivery class needs to create Courier objects. Instead of directly creating NationalCourier or InternationalCourier, use a factory method createCourier() that returns the right type based on context.

- **Pros:** Follows SRP (separates creation from usage) and OCP (add new types without modifying existing code)
- **Cons:** Code may become more complicated — many new subclasses needed

Builder Design Pattern

Problem: Creating objects with many optional parameters leads to 'telescoping constructors' — constructors with too many parameters.

Solution: Extract construction into a separate Builder class. Construct objects **step by step**, setting only the parameters you need.

Example: Book has isbn, title, cost (mandatory) + author, published, description (optional). Builder allows: Book.builder(isbn, title, cost).author('X').published(2024).build()

- **Pros:** Construct objects step by step; follows SRP
- **Cons:** New classes and methods needed

EXAM ALERT

Factory = CREATE objects without specifying exact class (factory decides which subclass).

Builder = CONSTRUCT complex objects STEP BY STEP (avoid telescoping constructors).

Key difference: Factory is about WHICH type to create. Builder is about HOW to construct with many parameters.

8.3.2 Structural Patterns

Facade Design Pattern

Problem: Need to create several objects, perform several steps, use complicated library functions. Others reusing your code must know all the details.

Solution: Provides a **simple interface** to a complex set of classes/library. Hides the complexity behind one easy-to-use class.

Example: After buyer places order: Create order → Notify seller → Prepare packaging → Send for delivery. A Facade wraps all these steps into one simple processOrder() method.

- **Pros:** Isolates code from complexity of other libraries/classes
- **Cons:** Facade becomes tightly coupled to underlying objects

Adapter Design Pattern

Problem: Two classes have **incompatible interfaces** and cannot work together directly.

Solution: Create an **Adapter** class that sits between them, converting one interface to another.

Example from slides: Product class has cost in rupees. ConversionCalculator has dollarsToRupee() and eurosToRupee(). Product can't directly use ConversionCalculator. Solution: CostCalculatorAdapter wraps ConversionCalculator and provides a getProductCost() method that Product can use.

- **Pros:** Follows SRP (separates conversion from business logic) and OCP (new adapter types without modifying existing code)
- **Cons:** Overall code complexity increases

EXAM ALERT

Facade vs Adapter — THE exam distinction:

- FACADE = SIMPLIFYING complexity (one simple method wrapping many steps)
- ADAPTER = making INCOMPATIBLE interfaces work together
- 'CoursePublisher with one method hiding verify+plagiarism+convert' = FACADE
- 'PaymentProcessor only handles USD but gateways provide EUR/INR' = ADAPTER
- 'Product cost in dollars, need rupees, incompatible classes' = ADAPTER

8.3.3 Behavioural Patterns

Iterator Design Pattern

Problem: Different collections (Lists, Queues, Stacks, Trees) store data differently. How to access and iterate through elements without knowing the internal structure?

Solution: Separate the traversal behaviour into a separate **Iterator** object with hasNext() and next() methods.

- **Pros:** Follows SRP and OCP. Client doesn't need to know how data is stored.
- **Cons:** Can be overkill for simple collections.

Observer Design Pattern

Problem: Need to notify a set of subscribers when something changes (e.g., notify buyers when new product launches).

Solution: Subject maintains a list of observers, notifies them automatically of changes. Observers register/unregister themselves.

- Subject has: registerObserver(), unregisterObserver(), notifyObservers()
- Each Observer implements: update()

ANALOGY

Think of a YouTube channel: you SUBSCRIBE (register), the channel NOTIFIES you when a new video drops, and you can UNSUBSCRIBE (unregister) anytime. The channel is the Subject, you are the Observer.

Strategy Design Pattern

Problem: Need to process something using different algorithms/strategies depending on context (e.g., process orders FIFO vs priority-based).

Solution: Extract different strategies into **separate classes**. The original class (context) delegates the work to a strategy object. Strategies are **interchangeable at runtime**.

- **Pros:** Isolate algorithm implementation; follows OCP — add new strategies without changing existing classes
- **Cons:** Not needed if there are only a few algorithms

EXAM ALERT

Design Pattern summary for exam matching:

- Creating objects without specifying class → FACTORY
- Constructing objects step by step → BUILDER
- Simplifying a complex subsystem → FACADE
- Making incompatible interfaces work → ADAPTER
- Traversing collections uniformly → ITERATOR
- Notifying subscribers of changes → OBSERVER
- Swapping algorithms at runtime → STRATEGY

Pattern	Category	Problem It Solves	Key Exam Identifier
Factory	Creational	Create objects without specifying exact class	'Factory method decides which subclass'
Builder	Creational	Construct complex objects step by step	'Many optional parameters', 'telescoping constructors'
Facade	Structural	Simplify complex subsystem with one interface	'One simple method hiding many steps'
Adapter	Structural	Make incompatible interfaces work together	'Incompatible', 'convert from one format to another'
Iterator	Behavioural	Traverse collections without knowing internals	'hasNext() and next()', 'different collections'

Observer	Behavioural	Notify subscribers when something changes	'Subscribe', 'notify', 'update on changes'
Strategy	Behavioural	Swap algorithms at runtime	'Different strategies', 'interchangeable algorithms'

EXAM PREP

Practice MCQs for Weeks 7 & 8

Q1. Which of the following is NOT an IDE?

- (a) VS Code
- (b) PyCharm
- (c) Thonny
- (d) TextEdit (Mac)

Answer: (d) TextEdit (Mac) -- Basic text editor — no debugger, no compiler, no project management

Q2. Third-party payment API not working. Engineer reviews API documentation and examples. Debugging strategy?

- (a) Input manipulation
- (b) Backward
- (c) Forward
- (d) Black box

Answer: (d) Black box -- 'Third-party' + 'documentation' = can't see inside = Black box

Q3. Error, Failure, Fault — which is the OBSERVABLE one?

- (a) Error
- (b) Failure
- (c) Fault
- (d) Bug

Answer: (b) Failure -- Failure = observable error (wrong output, exception, crash)

Q4. In PDB, which command steps INTO a function call?

- (a) n (next)
- (b) s (step)
- (c) c (continue)
- (d) p (print)

Answer: (b) s (step) -- n = step OVER, s = step INTO

Q5. for loop + if/else inside + another if. Cyclomatic complexity?

- (a) 2
- (b) 3
- (c) 4
- (d) 5

Answer: (c) 4 -- for(1)+if(1)+if(1)=3. CC=3+1=4. else NOT counted!

Q6. Code has repeated if month=='April' or month=='June'... blocks. Which principle is violated?

- (a) SOLID
- (b) DRY
- (c) YAGNI
- (d) KISS

Answer: (b) DRY -- Don't Repeat Yourself — extract into a list/method

Q7. Changing code structure WITHOUT changing behaviour is called:

- (a) Debugging
- (b) Refactoring
- (c) Rewriting
- (d) Optimizing

Answer: (b) Refactoring -- Refactoring = improve structure, NOT change behavior

Q8. Seller class manages seller data AND products AND inventory. SOLID principle violated?

- (a) Open-Closed
- (b) Interface Segregation
- (c) Single Responsibility
- (d) Dependency Inversion

Answer: (c) Single Responsibility -- Class with multiple responsibilities = SRP violation

Q9. SmartLight forced to implement play_music(). SOLID principle violated?

- (a) SRP
- (b) OCP
- (c) LSP
- (d) ISP

Answer: (d) ISP -- 'Forced to implement unused methods' = ALWAYS ISP

Q10. Seller depends directly on BankAccount. Adding PayTM/UPI requires changes. SOLID violation?

- (a) SRP
- (b) OCP
- (c) ISP
- (d) DIP

Answer: (d) DIP -- Should depend on PaymentAccount interface, not BankAccount directly

Q11. Adding Clothing product type requires modifying existing Product class. SOLID principle?

- (a) SRP
- (b) OCP
- (c) LSP
- (d) ISP

Answer: (b) OCP -- Should extend (create subclass), not modify existing class

Q12. CoursePublisher with one publish() method hiding verify, plagiarism check, video convert, catalog update. Pattern?

- (a) Builder
- (b) Adapter
- (c) Facade
- (d) Strategy

Answer: (c) Facade -- Simple interface hiding complex multi-step operations

Q13. PaymentProcessor (USD) needs to work with gateways providing EUR/INR. Pattern?

- (a) Factory
- (b) Builder
- (c) Adapter
- (d) Facade

Answer: (c) Adapter -- Making incompatible interfaces work together

Q14. Book object with isbn, title (mandatory) + author, description (optional). Avoid telescoping constructors. Pattern?

- (a) Factory
- (b) Builder
- (c) Adapter
- (d) Observer

Answer: (b) Builder -- Complex object with many optional params = Builder

Q15. Delivery class creates NationalCourier or InternationalCourier based on context. Pattern?

- (a) Factory
- (b) Builder
- (c) Strategy
- (d) Observer

Answer: (a) Factory -- Factory method decides WHICH subclass to create

Q16. Process orders using FIFO or Priority depending on user type. Algorithms swappable at runtime. Pattern?

- (a) Factory
- (b) Observer
- (c) Strategy
- (d) Iterator

Answer: (c) Strategy -- Interchangeable algorithms at runtime = Strategy

Q17. Notify buyers when new product launches. Buyers can subscribe/unsubscribe. Pattern?

- (a) Strategy
- (b) Observer
- (c) Iterator
- (d) Facade

Answer: (b) Observer -- Subscribe + notify on changes = Observer

Q18. Traverse List, Queue, Stack uniformly without knowing internal structure. Pattern?

- (a) Adapter
- (b) Observer
- (c) Iterator
- (d) Strategy

Answer: (c) Iterator -- Traverse different collections uniformly = Iterator

Q19. File-sharing app, no central server, users share storage and files directly. Architecture?

- (a) Client-Server
- (b) MVC
- (c) Pipe and Filter
- (d) Peer-to-Peer

Answer: (d) Peer-to-Peer -- 'No central server' + 'share directly' = P2P

Q20. In MVC, which component is the visual representation of data?

- (a) Model
- (b) View
- (c) Controller
- (d) Connector

Answer: (b) View -- View = GUI/presentation layer, visual representation

Q21. def func(): for + while + if + elif + else. Cyclomatic complexity?

- (a) 3
- (b) 4
- (c) 5
- (d) 6

Answer: (c) 5 -- for(1)+while(1)+if(1)+elif(1)=4. CC=4+1=5. else NOT counted!

Q22. A function has flag argument controlling what it does. This is a:

- (a) Code smell
- (b) Design pattern
- (c) Refactoring technique
- (d) Testing strategy

Answer: (a) Code smell -- Flag arguments are a function code smell

CHEAT SHEET

Week 7 & 8 Quick Revision

Week 7: Development & Debugging

IDE = editor + compiler + debugger + project management | Text editor = just text editing
TextEdit (Mac) = NOT an IDE | Thonny = IS an IDE | VS Code = IS an IDE
Framework = bundled common features for quick development (e.g., Flask for web apps)
Error → Failure (observable) → Fault (location) → Debugging (finding cause)
Debug strategies: Input Manipulation | Backwards (from error) | Forwards (from start) | Black Box (3rd-party, use docs)
'Third-party' + 'can't see code' + 'documentation' = BLACK BOX — appears in EVERY paper!
PDB commands: n(next/over) | s(step/into) | p(print) | b(break) | c(continue)

Week 7: Metrics & Clean Code

CC = Decision Points + 1 | Decisions: if, elif, for, while, except, and, or | NOT: else
CC Scores: 1-5(A,low) | 6-10(B,stable) | 11-20(C,moderate) | 21-30(D,complex) | 31+(E/F,alarming)
Raw Metrics: LOC (total lines) | LLOC (logical lines) | SLOC (source lines) | Comments
Halstead: $n = n_1 + n_2$ (vocabulary) | $N = N_1 + N_2$ (length) | $V = N \times \log_2(n)$ | $D = (n_1/2) \times (N_2/n_2)$ | $E = D \times V$
Code Smells: redundant comments | too-long functions | too-many args | flag args | dead functions | DRY violations
Refactoring = improve structure WITHOUT changing behavior | Tools: Pylint, flake8, vulture

Week 8: Architecture

Architecture Styles: Client-Server | Pipe & Filter | MVC | Peer-to-Peer
MVC: Model(data/DB) | View(GUI/presentation) | Controller(logic/coordination)
P2P: NO central server, nodes share directly, each is client+server
Component = well-defined functionality | Connector = transmits info between components | Protocol = rules for interaction

Week 8: SOLID Principles

S (SRP): One class = ONE reason to change | Violation: class doing too many things
O (OCP): Open for EXTENSION, closed for MODIFICATION | Fix: use inheritance/subclasses
L (LSP): Subtypes SUBSTITUTABLE for base types | Fix: design proper abstractions
I (ISP): Don't FORCE unused methods | 'Forced to implement' = ALWAYS ISP | Fix: split fat interfaces
D (DIP): Depend on ABSTRACTIONS not concretions | Fix: use interfaces (PaymentAccount not BankAccount)

Week 8: Design Patterns

Creational: Factory (which class to create) | Builder (step-by-step construction, many optional params)
Structural: Facade (simplify complex subsystem) | Adapter (incompatible interfaces)
Behavioural: Iterator (traverse collections) | Observer (subscribe/notify) | Strategy (swap algorithms)
FACADE = simplifying | ADAPTER = incompatible — KEY exam distinction!
Factory follows SRP+OCP | Builder follows SRP | Facade isolates complexity | Observer = pub/sub pattern
Design smells → use SOLID → refactor if needed | Patterns have Pros AND Cons — not used blindly

Made by Anmol Kansal with AI help

Software Engineering — IIT Madras BS Degree Programme

■ **Best of luck with your exam!** ■

SOFTWARE ENGINEERING

Week 9, 10 & 11 — Detailed Notes

IIT Madras BS Degree Programme

- Week 9: Software Testing — Unit, Black Box, White Box, Integration, System, TDD
 - Week 10: Deployment & Monitoring — Environments, Strategies, Hosting, CI/CD, Performance
 - Week 11: Software Organizations, Communication, Productivity, Ethics, Decision Making
 - 25+ Practice MCQs from Both NEW Previous Year Papers (Apr 2025 & Dec 2024)
 - Exam Alerts, Diagrams, Tips & Cheat Sheet
-

Made by Anmol Kansal with AI help

WEEK 9

Software Testing

Topics Covered This Week

1. Importance of Testing, Test Cases, Test Suites
2. Black Box Testing: Equivalence Class Partitioning, Boundary Value Analysis
3. White Box Testing: Branch Coverage, Multiple Condition Coverage, Path Coverage
4. Unit Testing: Drivers, Stubs, pytest/unittest
5. Integration Testing: Big Bang, Top-Down, Bottom-Up, Mixed
6. System Testing: Alpha, Beta, Acceptance, Smoke, Stress/Performance
7. Test-Driven Development (TDD): Red-Green-Refactor cycle

9.1 Why Testing Matters

Software errors can cause entire systems to crash — airport systems, spacecraft, medical equipment. Even simple bugs can have catastrophic consequences. In 2002, software bugs cost the US economy \$59.5 billion.

Test Cases & Test Suites

A **Test Case** is a triplet **[I, S, R]** where: **I** = input data, **S** = state of the program when data is input, **R** = expected result.
A **Test Suite** = set of all test cases designed to test a program.

Why **design** test cases? Testing a large collection of **randomly selected** test cases does NOT guarantee all errors will be found. You need to choose test cases **carefully** to cover different types of errors.

ANALOGY

Testing with random inputs is like checking if a bridge is safe by only walking across the middle. You also need to test the edges, heavy loads, and extreme weather. Similarly, testing needs systematic design.

9.2 Levels of Testing

Level	What Is Tested	Who Does It	When
Unit Testing	Individual functions/units in isolation	Developer writing the code	DURING coding (not testing phase!)
Integration Testing	How units work TOGETHER, detect interface errors	Dev/QA team	After units are unit-tested
System Testing	Fully integrated system as a whole	QA team / testers	After integration testing
Acceptance Testing	Does system meet CLIENT requirements?	Customer/stakeholders	Before final delivery

EXAM ALERT

Unit testing happens DURING coding (by the developer), NOT in the testing phase!
Alpha = internal team | Beta = external real users | Acceptance = customer decision
'Fully integrated system tested by select group of customers' = BETA testing (from PYQ Dec 2024)

9.3 Black Box Testing

Treat the program as a **black box** — examine ONLY input/output values. **No knowledge of code or design required.**

9.3.1 Equivalence Class Partitioning

Partition the input domain into **equivalence classes** — groups where the program behaves similarly for every input in that class. Design ONE test case per class.

Example 1: `min(x, y)` function → Equivalence classes: $(x < y)$, $(x > y)$, $(x = y)$

Example 2: `isPrime(num)` → Classes: Prime numbers, Non-prime numbers

Example 3: `determineSign(num)` → Classes: Negative ($-\infty$ to -1), Zero (0), Positive (1 to ∞)

EXAM ALERT

PYQ Apr 2025: 'determineSign(int num) classifies as positive, negative, or zero. Equivalence classes?'

Answer: $(-\infty$ to -1), (0), (1 to ∞) — three distinct behaviors, three classes.

DON'T combine 0 with negatives or positives!

9.3.2 Boundary Value Analysis

Examine values at the **boundaries** of equivalence classes. Bugs often occur at edges (off-by-one errors).

Example: `isPrime(num)` → Boundary values: 0, 1 (edge cases between prime/non-prime)

Example: for i in range(n) vs range(n-1) → Test n, n-1, n+1

REMEMBER

Black Box Summary:

1. From requirements, identify input/output structures
2. Identify equivalence classes
3. Pick ONE representative test from each class
4. Add boundary value test cases

9.4 White Box Testing

Analyze the **structure of the program** (you CAN see the code). Focus on **coverage** — ensuring all paths are tested.

Coverage Type	What Must Be Covered	Test Cases Needed	Example
Branch Coverage	Every BRANCH taken at least once	Both true and false paths of every if/else	<code>min(x,y)</code> : need $x < y$ AND $x > y$
Multiple Condition Coverage	Each component in composite conditions takes T and F	All combinations of sub-conditions	<code>if(num < 50 num > 150)</code> : need {49, 100, 200}
Path Coverage	Every linearly independent PATH at least once	All unique paths through control flow graph	Number of paths = Cyclomatic Complexity

Branch Coverage vs Multiple Condition Coverage

Example: `if(num < 50 || num > 150)`

- **Branch coverage:** {49, 100} — covers true branch ($49 < 50$) and false branch (100 is between 50-150)
- **Multiple condition coverage:** {49, 100, 200} — also covers $\text{num} > 150$ being true (200)

EXAM ALERT

PYQ: Given code with 'if age < 18 or age > 60', find test cases for branch vs multiple condition coverage.

Branch coverage: need at least one test making condition TRUE, one making it FALSE.

Multiple condition: need each sub-condition ($\text{age} < 18$ AND $\text{age} > 60$) to be individually true and false.

So branch might need {10, 25, 45} but multiple condition needs {10, 25, 45, 65} to cover $\text{age} > 60$ being true.

Path Coverage

Number of linearly independent paths = **Cyclomatic Complexity**. Use Control Flow Graph to identify all paths, then design test cases for each path.

REMEMBER

White Box requires access to SOURCE CODE.

Black Box requires only SPECIFICATIONS (no code needed).

Branch < Multiple Condition < Path coverage (increasing strictness).

PYQ: 'Designing test cases necessitates access to source code' = TRUE for White Box, FALSE for Black Box.

9.5 Unit Testing — Details

Unit testing tests individual modules in isolation. It happens **during coding**, done by the **person writing the code**. Why not just integrate and test everything at once? Because if a failure is detected during integration, you can't tell WHICH module has the error.

Testing Environment

- **Driver Module:** Contains non-local data structures accessed by the module under test. SIMULATES the calling module.
- **Stub:** Dummy procedure that has the same I/O parameters as the function called by the unit under test, but has a **highly simplified behaviour**. SIMULATES called modules.

EXAM ALERT

Driver = simulates the CALLER (calling the function under test).

Stub = simulates the CALLED function (functions that the unit calls).

Top-down testing needs STUBS (testing from top, lower modules not ready).

Bottom-up testing needs DRIVERS (testing from bottom, upper modules not ready).

9.6 Integration Testing Approaches

Approach	How It Works	Advantage	Disadvantage	Needs
Big Bang	ALL modules integrated in a single step	Simple for small systems	Difficult to localize errors	Nothing extra
Bottom-Up	Start from leaf modules, integrate upward	No stubs needed	Drivers required	DRIVERS
Top-Down	Start from root, add subordinate modules	No drivers needed	Stubs required	STUBS
Mixed	Combine top-down and bottom-up	Flexible, practical	More complex to manage	Both

EXAM ALERT

PYQ Apr 2025: 'Characteristic of Big Bang approach?' → 'All modules integrated in a single step'

NOT 'errors easy to localize' (that's the DISADVANTAGE of Big Bang)

NOT 'modules integrated incrementally' (that's top-down/bottom-up)

9.7 System Testing Types

Type	What It Tests	Key Identifier
Alpha Testing	Full system by INTERNAL team within the org	'Test team within the organisation'
Beta Testing	Full system by EXTERNAL select group of customers	'Select group of customers'

Acceptance Testing	Customer decides whether to ACCEPT delivery	'Customer determines whether to accept'
Smoke Testing	Basic functionalities BEFORE initial system testing	'Basic functionalities working?', 'few basic test cases'
Stress/Performance Testing	System pushed BEYOND expected capacity	'Beyond designed capacity', 'simulate many transactions'

EXAM ALERT

PYQ Apr 2025: 'Push system beyond expected capacity to see how it handles high loads' = STRESS Testing
 PYQ Dec 2024: 'Fully integrated system tested by selected group of customers' = BETA Testing
 Smoke testing = quick sanity check BEFORE detailed testing (does it even turn on?)
 Performance testing checks NON-FUNCTIONAL requirements (response time, throughput)

9.8 Test-Driven Development (TDD)

Used especially in **Agile** processes. The core cycle is **RED** → **GREEN** → **REFACTOR**:

- **RED:** Write a test for functionality you want → Run the test → See it FAIL (no code exists yet)
- **GREEN:** Write the MINIMUM code to make the test pass → Run it → See it PASS
- **REFACTOR:** Improve code quality — make it modular, elegant, clean — without changing behavior

REMEMBER

TDD = Write tests FIRST, then code. Not the other way around.
 Drive design and development FROM tests.
 Refactor = improve structure without changing behavior (same as Week 7 definition).

WEEK 10

Software Deployment & Monitoring

Topics Covered This Week

1. Deployment Environments: Development, Testing, Staging, Production
2. Deployment Strategies: Blue/Green, Canary, Versioned
3. Hosting Options: Bare Metal, IaaS, PaaS
4. Continuous Integration (CI): Best Practices, CI Servers (Jenkins)
5. Performance: Caching, Task Queues, Minification, Compression
6. Monitoring: Analytics, Clickstreams, A/B Testing

10.1 Deployment Environments

An **environment** is a system that executes a software application. Software passes through four environments:

Environment	What It Is	Key Detail
Development	Local environment of each developer	Contains IDE, local DB, working copy of code
Testing	Shared environment where integrated code is tested	Multiple developers' code merged and tested together
Staging	EXACTLY resembles production, runs on remote machine	Preview features, performance test BEFORE going live
Production	Live system used by actual users	The real deal — downtime here affects users

Why Staging? Deploying involves many configurations — if something goes wrong, the live system can crash. Staging lets you preview, test performance, and catch issues safely.

EXAM ALERT

PYQ Apr 2025: 'Characteristics of DEVELOPMENT environment?'

Answer: Contains IDE and tools + allows developers to write and debug code

NOT 'exactly mimics production' (that's STAGING)

NOT 'used for live user interactions' (that's PRODUCTION)

10.2 Deployment Strategies

Strategy	How It Works	Advantage	Disadvantage	Key Identifier
Blue/Green	Create NEW separate production env for new version, switch traffic over	Easy ROLLBACK (switch back to old env)	Needs double infrastructure	'Staged deployment', 'rollback is easy'
Canary	Slowly roll out to SMALL SUBSET of users (by region, demographics, randomly)	Low risk — test on few before all	Manage multiple instances	'Phased/incremental rollout', 'small subset'
Versioned	Keep ALL versions alive, let users CHOOSE which version	User freedom of choice	Must maintain multiple versions	'Users choose version', 'all versions alive'

EXAM ALERT

PYQ Apr 2025: 'Release to small percentage based on location and demographics' = CANARY
PYQ Dec 2024: 'Keeps all versions live, users freely choose' = VERSIONED
Blue/Green = instant switch + easy rollback (like a light switch)
Canary = gradual rollout (like a canary in a coal mine — test with few first)

10.3 Hosting Options

Option	What You Get	Cost	Control	Examples
Bare Metal	Actual physical server hardware	Most expensive upfront	FULL control	Purchase own servers
IaaS	Virtual machines (sliced from physical servers)	Cheaper, pay-as-you-go	Good control	AWS, Digital Ocean, Linode
PaaS	Software layer + infrastructure (just deploy your app)	Easiest, lowest effort	LIMITED control	Heroku, Google App Engine

EXAM ALERT

PYQ Dec 2024: 'Highest performance + full control of hardware' = BARE METAL
PYQ Apr 2025: 'PaaS disadvantages?' = Lack of control + limited customization vs IaaS
IaaS disadvantage: shared with others (performance may suffer) + own configurations needed
PaaS advantage: VERY easy to deploy

10.4 Continuous Integration (CI)

CI = **automating the integration and deployment** of software. Developer commits → CI server pulls code → builds and tests → deploys to staging/production.

7 Best Practices for CI

- 1. **Maintain a single source repository** (one codebase)
- 2. **Automate the Build** (tools: Ant, Gradle)
- 3. **Make the Build Self-Testing** (automated tests, notify on failure)
- 4. **Commit to mainline EVERY DAY** (frequent commits = fewer conflicts)
- 5. **Every commit builds mainline on integration server** (CI server monitors repo)
- 6. **Fix broken builds IMMEDIATELY** (roll back to last-known good build)
- 7. **Automate Deployment** (write deployment scripts)

Benefits: Reduces deployment time, avoids last-minute rush, early prototypes for demos, pushes modular code.

Drawback: Initial effort required to set up processes.

EXAM ALERT

PYQ Dec 2024: 'Major DRAWBACK of CI?' = 'Initial effort required to set up the process'
NOT 'long deployment time' (CI REDUCES deployment time)
NOT 'last-minute confusion' (CI AVOIDS this)
CI server examples: Jenkins, Cruise Control

10.5 Performance & Monitoring

Performance Techniques

- Caching:** Store results of common operations in memory (Memcached, Redis). Levels: browser, web server, DB query cache.
- Task Queues:** Heavy tasks (emails, bulk updates) executed OUTSIDE the HTTP request-response cycle. Queue of jobs + pool of workers.

- **Minification:** Remove unnecessary characters from code. E.g., 68B → 40B (41% saving).
- **Compression:** Compress static pages with Brotli, Gzip.

Impact: AliExpress sped up site by 30% → orders jumped 10.5%. Google slowed search by 0.5s → requests dropped 25%.

EXAM ALERT

PYQ Apr 2025: 'Improve page speed and increase sales?' = 'Minify and compress static content'

PYQ Dec 2024: 'Brotli and Gzip' = Loading and rendering pages (compression)

NOT 'reduce caching' (more caching = better, not less!)

Monitoring & Analytics

- **System monitoring:** CPU utilization, memory, storage, network bandwidth, latency
- **Tools:** Lighthouse (performance score, SEO, accessibility)
- **Customer behavior:** Clickstreams (page sequences), Dwell time (time on page), Abandonment rates
- **A/B Testing:** Version A vs Version B → Canary deployment → measure which performs better (traffic, sales, dwell time)

WEEK 11

Other Aspects in Software Engineering

Topics Covered This Week

1. Software Organization Roles: Marketing, Product Manager, Designers, Engineers, Sales, Support, Data Scientists, Ethics
2. Conceptual Integrity & Communication Tools (JIRA, GitHub, Slack, Stack Overflow)
3. Productivity: Project Management Tools, Knowledge Sharing, Cross-training
4. Qualities of Great Software Engineers (Microsoft study)
5. Decision Making: Macro vs Micro decisions, Rational Decision Process

11.1 Roles in a Software Organization

Role	Responsibility	Key Detail for Exam
Marketing Team	Look for market opportunities, conduct research, identify audiences	'Understand target audience and market demand' = Marketing
Product Manager	Intersection of business + technology + user. Maximizes ROI	Knows tech stack, understands effort, works with marketing AND sales
Designers (UX)	Transform requirements to solutions, talk to users, create prototypes	User Experience focus
Software Engineers	Write code to implement requirements	Usually DON'T decide what product is made
Engineering/Project Manager	Organize work, coordinate teams, resolve conflicts	Also known as project manager; NOT same as product manager
Sales Team	Sell product to users marketing has identified	Provide feedback to product and design teams
Support Team	Resolve client problems, report defects	Feedback to product, design, engineering about shortcomings
Data Scientists	Analyze data, help org make better decisions	Help marketing, sales, engineers understand patterns
Ethics/Policy	Shape terms of service, licenses, privacy policy	Background in law, social science, policy

EXAM ALERT

PYQ Apr 2025: 'Understand target audience and market demand before app launch. Which team?' = MARKETING
PYQ Dec 2024: 'Primary tasks of project manager?' = Coordinating teams + Resolving conflicts
NOT 'conduct market research' (that's marketing)
NOT 'provide feedback about product defects' (that's support)

11.2 Communication & Knowledge Sharing

Conceptual Integrity = everyone on the team has the **same understanding** of what is being built and why. Effective communication ensures this.

Tool Type	Examples	Purpose
Issue Trackers	JIRA, Pivotal Tracker	Track issues, history of who worked on what
Project Management	JIRA, Pivotal Tracker	High-level AND detailed view of progress
Code Hosting	GitHub Pages	Host libraries, frameworks, documentation

Communication	Slack	Real-time team communication
Q&A / Knowledge	Stack Overflow	Resolve issues, additional learning resources

EXAM ALERT

PYQ Apr 2025: 'Tool for high-level overview AND detailed view of progress?' = Project management tools (JIRA)

PYQ Dec 2024: 'Reuse knowledge across projects, documented and archived?' = Knowledge-sharing tools
Knowledge is lost when people LEAVE the org → mitigation = 'cross-training' (rotating devs between projects)

11.3 Productivity

Productivity in SE is NOT just lines of code per day. A developer who delivers features but ignores teammates harms **team** productivity.

- **Right tools matter:** Project management tools (JIRA), IDEs with features (auto-completion, refactoring)
- **Cross-training:** Rotating developers between projects mitigates knowledge loss when people leave

11.4 Decision Making

- **Macro decisions:** Design and architecture level (which libraries, frameworks to use)
- **Micro decisions:** Algorithms, data structures for a particular module

Rational Decision Making Process: 1. Identify decision → 2. Identify alternatives → 3. Think through outcomes → 4. Evaluate best outcome for context → 5. Make decision

EXAM PREP

Practice MCQs for Weeks 9-11 (from PYQs)

MCQs from PYQ Apr 2025 (QIM2)

Q1. Startup wants to understand target audience and market demand. Which team leads?

- (a) Designers
- (b) Software Engineers
- (c) Marketing Team
- (d) Sales Team

Answer: (c) Marketing Team -- Market research = Marketing

Q2. Tool for high-level overview AND detailed view of each member's progress?

- (a) IDEs
- (b) Slack
- (c) JIRA/Pivotal Tracker
- (d) Stack Overflow

Answer: (c) JIRA/Pivotal Tracker -- Project management tools

Q3. Pages too slow, sales dropping. Best strategy?

- (a) Minify and compress
- (b) Increase page elements
- (c) Load unnecessary JS first
- (d) Reduce caching

Answer: (a) Minify and compress -- Minification + compression = faster pages

Q4. Characteristic of Big Bang approach?

- (a) Modules integrated incrementally
- (b) Errors easy to localize
- (c) All modules integrated in single step
- (d) Drivers required

Answer: (c) All modules integrated in single step -- Big Bang = everything at once

Q5. Push system beyond expected capacity, high data loads. Testing type?

- (a) Smoke
- (b) Stress
- (c) Beta
- (d) Acceptance

Answer: (b) Stress -- Beyond designed capacity = Stress testing

Q6. Organize historical weather data efficiently. Design aspect?

- (a) Components
- (b) Interfaces
- (c) Data structures
- (d) Algorithms

Answer: (c) Data structures -- Organizing and accessing data = data structures

Q7. Mathematical models and predefined formulas for estimation (not human judgement)?

- (a) Empirical
- (b) Delphi
- (c) Expert Judgement
- (d) Heuristic

Answer: (d) Heuristic -- COCOMO uses formulas = Heuristic estimation

Q8. Release to small % based on location/demographics?

- (a) Blue/Green
- (b) Versioned
- (c) Canary
- (d) None

Answer: (c) Canary -- Small subset + demographics = Canary

Q9. determineSign(num): equivalence classes?

- (a) (-inf to -1), (0), (1 to inf)
- (b) (-inf to 0), (1 to inf)
- (c) (-20 to -1), (0 to 1), (2 to inf)
- (d) Negative only, Positive only

Answer: (a) (-inf to -1), (0), (1 to inf) -- Three behaviors: negative, zero, positive

Q10. Unified interface hiding create order + notify + package + deliver?

- (a) Facade
- (b) Adapter
- (c) Observer
- (d) Strategy

Answer: (a) Facade -- Single interface hiding complex steps = Facade

Q11. Traces back from wrong output to find original mistake. Strategy?

- (a) Input manipulation
- (b) Backwards
- (c) Forwards
- (d) Blackbox

Answer: (b) Backwards -- From error trace back = Backwards

Q12. Shared global variable for total stock, all modules affected. Coupling?

- (a) Data
- (b) Control
- (c) Common
- (d) Content

Answer: (c) Common -- Shared global data = Common coupling

Q13. Frequent travelers struggle to navigate redesigned site. Usability factor?

- (a) Learnability
- (b) Effectiveness
- (c) Memorability
- (d) Safety

Answer: (c) Memorability -- RETURNING users forgetting how to use = Memorability

Q14. Add to Cart always bottom-right, but one page has it top-left. Heuristic?

- (a) Consistency
- (b) Recognition over Recall
- (c) Show Status
- (d) Flexibility

Answer: (a) Consistency -- Inconsistent placement = Consistency violated

MCQs from PYQ Dec 2024 (QDB2)

Q15. Fully integrated system tested by selected group of customers?

- (a) Integration

- (b) Acceptance
- (c) Alpha
- (d) Beta

Answer: (d) Beta -- Select group of customers = Beta

Q16. Keeps all versions live, users choose version?

- (a) Blue/Green
- (b) Canary
- (c) Versioned
- (d) None

Answer: (c) Versioned -- All versions alive + user choice = Versioned

Q17. Major drawback of continuous integration?

- (a) Long deployment time
- (b) Initial setup effort
- (c) Last-minute rush
- (d) Late prototypes

Answer: (b) Initial setup effort -- CI's ONE drawback = initial setup

Q18. Highest performance + full control of hardware?

- (a) Bare metal
- (b) IaaS
- (c) PaaS
- (d) SaaS

Answer: (a) Bare metal -- Physical servers = maximum performance + control

Q19. Brotli and Gzip improve web app performance. Category?

- (a) Caching
- (b) Task queues
- (c) Loading/rendering pages
- (d) Monitoring

Answer: (c) Loading/rendering pages -- Compression for faster page loading

Q20. Reuse knowledge across projects, needs documentation?

- (a) Project management
- (b) Development tools
- (c) Knowledge-sharing tools
- (d) None

Answer: (c) Knowledge-sharing tools -- Documented + archived knowledge

Q21. Flag passed to delivery module decides express vs normal. Coupling?

- (a) Data
- (b) Control
- (c) Common
- (d) Content

Answer: (b) Control -- Flag controlling behavior = Control

Q22. Desired characteristics of good software design?

- (a) Low cohesion, low coupling
- (b) Low cohesion, high coupling
- (c) High cohesion, low coupling
- (d) High cohesion, high coupling

Answer: (c) High cohesion, low coupling -- Golden rule of design

Q23. Code quality based on number of lines of source code?

- (a) Cyclomatic Complexity

-
- (b) Raw Metrics
 - (c) Halstead's
 - (d) All

Answer: (b) Raw Metrics -- LOC, LLOC, SLOC = Raw Metrics

Q24. Go through code line by line, inspect variables in debugger?

- (a) Logging
- (b) Dump and diff
- (c) Stepping in debugger
- (d) Profiling

Answer: (c) Stepping in debugger -- Line-by-line in debugger = Stepping

Q25. DIP applied: DeliveryManager uses Vehicle interface instead of CompanyVehicle directly. SOLID?

- (a) OCP
- (b) LSP
- (c) ISP
- (d) DIP

Answer: (d) DIP -- Depend on interface (abstraction) not concrete class

Q26. Unified class for invoice: format + validate + print + track. Single method call. Pattern?

- (a) Builder
- (b) Facade
- (c) Adapter
- (d) Strategy

Answer: (b) Facade -- Single interface hiding multi-step complexity

CHEAT SHEET

Week 9-11 Quick Revision

Week 9: Testing

Test Case = [Input, State, Expected Result] | Test Suite = set of all test cases

Black Box: no code needed | Equivalence classes + Boundary values | Just input/output

White Box: need source code | Branch coverage < Multiple condition < Path coverage

Branch: every branch taken once | Multiple condition: each sub-condition T and F | Path: all independent paths

Unit test: DURING coding by developer | Drivers simulate callers | Stubs simulate called functions

Integration: Big Bang (all at once) | Bottom-Up (needs drivers) | Top-Down (needs stubs) | Mixed

System: Alpha(internal) | Beta(external customers) | Smoke(basic sanity) | Stress(beyond capacity)

TDD: RED(write test, fail) -> GREEN(write code, pass) -> REFACTOR(improve structure)

Week 10: Deployment & Monitoring

Environments: Development(local) -> Testing(shared) -> Staging(resembles production) -> Production(live)

Staging EXACTLY resembles production | Used for preview + performance testing before going live

Blue/Green: new env, switch traffic, easy ROLLBACK | Canary: small subset first, phased rollout

Versioned: all versions alive, users CHOOSE | Drawback: maintain multiple versions

Bare Metal: highest performance, full control, most expensive | IaaS: VMs, cheaper | PaaS: easiest, least control

CI best practices: single repo, automate build, self-testing, daily commits, fix broken builds IMMEDIATELY

CI drawback: initial effort to setup | CI benefit: reduces deployment time, avoids last-minute rush

Performance: Caching(Memcached/Redis) | Task Queues(async) | Minify+Compress(Brotli/Gzip)

A/B Testing: Canary deploy Version A vs B, measure which is better (traffic, sales, dwell time)

Week 11: Organizations & More

Marketing = market research, identify audiences | Product Manager = business + tech + user intersection

Engineers usually DON'T decide what product is made | Project Manager = coordinate + resolve conflicts

Conceptual Integrity = everyone understands WHAT is being built and WHY

Knowledge sharing: JIRA(issues), Slack(communication), GitHub(code), Stack Overflow(Q&A;)

When people leave, knowledge goes with them -> Mitigation: cross-training (rotate devs between projects)

Productivity != lines of code | Right tools: JIRA for PM, IDEs for development

Decisions: Macro(architecture, frameworks) vs Micro(algorithms, data structures for a module)

Made by Anmol Kansal with AI help

Software Engineering — IIT Madras BS Degree Programme

Best of luck with your exam!